Contents

Event Bus **8**

1. Overview 8

Key Features 8

Design Principles 8

2. API Reference 8

2.1. `EventBus.on(event_name, callback)` 8

2.2. `EventBus.emit(event_name, data)` 10

2.3. `EventBus.off(event_name, callback)` 11

2.4. `EventBus.once(event_name, callback)` 11

2.5. `EventBus.clear(event_name)` 12

2.6. `EventBus.clear_all()` 13

2.7. `EventBus.has(event_name)` 13

3. Internal Implementation 13

3.1. Listener Storage 14

3.2. Shared Module State 14

3.3. Event Dispatch 14

4. Limitations 14

5. Quick Reference 15

6. Usage Guidelines 15

7. Summary 15

Timer **17**

1. Overview 17

Key Features 17

Design Principles 17

2. API Reference 17

2.1. `Timer.after(delay, callback, data)` 17

2.2. `Timer.every(interval, callback, data)` 18

2.3. `Timer.cancel(timer_id)` 19

2.4. `Timer.cancel_all()` 20

3. Internal Implementation 20

3.1. Timer Tracking 21

3.2. Integration with Defold 21

3.3. Shared Module State 21

3.4. Timer Lifecycle 21

4. Limitations 22

5. Quick Reference 22

6. Usage Guidelines 22

7. Summary 23

State Machine **24**

1. Overview 24

Key Features 24

Design Principles 24

2. API Reference 24

2.1. `StateMachine.new(initial_state)` 25

Table of Content

2.2. `StateMachine.add(machine, name, on_enter, on_exit, on_update)` 25
2.3. `StateMachine.set(machine, new_state)` 26
2.4. `StateMachine.get(machine)` 27
2.5. `StateMachine.remove(machine, name)` 28
2.6. `StateMachine.has(machine, name)` 29
2.7. `StateMachine.update(machine, dt)` 29
2.8. `StateMachine.is(machine, state_name)` 30
3. Internal Implementation 30
3.1. State Machine Structure 30
3.2. Independent Instances 31
3.3. State Registration and Activation 31
3.4. State Transition Lifecycle 31
4. Limitations 31
5. Quick Reference 32
6. Usage Guidelines 32
7. Summary 32
Input Bindings 34
1. Overview 34
Key Features 34
Design Principles 34
2. API Reference 34
2.1. `Input.new()` 34
2.2. `Input.bind(input, action_name, key)` 35
2.3. `Input.unbind(input, action_name)` 36
2.4. `Input.get_binding(input, action_name)` 36
2.5. `Input.has(input, action_name)` 37
2.6. `Input.clear(input)` 37
3. Internal Implementation 38
3.1. Data Structure 38
3.2. Instance Independence 38
3.3. Binding Replacement 38
4. Limitations 38
5. Quick Reference 39
6. Usage Guidelines 39
7. Summary 39
Save System 40
1. Overview 40
Key Features 40
Design Principles 40
2. API Reference 40
2.1. `Save.new()` 40
2.2. `Save.set(save, key, value)` 41
2.3. `Save.get(save, key)` 42
2.4. `Save.remove(save, key)` 42
2.5. `Save.save_to_file(save, filename)` 43

Table of Content

2.6. Save.load_from_file(save, filename) 43
2.7. Save.exists(filename) 44
3. Internal Implementation 45
3.1. Data Structure 45
3.2. In-Memory Storage 45
3.3. Persistence Through Defold 46
3.4. Loading Behavior 46
4. Limitations 46
5. Quick Reference 46
6. Usage Guidelines 47
7. Summary 47
Ability System 48
1. Overview 48
2. Main Features 48
3. API Reference 48
3.1 Ability.new(name, on_use) 48
3.2 Ability.unlock(ability) 49
3.3 Ability.lock(ability) 49
3.4 Ability.enable(ability) 49
3.5 Ability.disable(ability) 49
3.6 Ability.can_use(ability) 49
3.7 Ability.use(ability, data) 50
3.8 Ability.stop(ability) 50
3.9 Ability.set_cooldown(ability, duration) 51
3.10 Ability.update(ability, dt) 51
3.11 Ability.get_cooldown(ability) 51
3.12 Ability.get_cooldown_remaining(ability) .. 51
4. Typical Usage Example 52
5. Implementation Notes 52
5.1 Ability Instances 52
5.2 Availability Rules 52
5.3 Active State 52
5.4 Cooldown Management 53
5.5 Timer Dependency 53
5.6 Error Handling 53
6. Limitations 53
7. Quick Reference 53
8. Usage Guidelines 54
9. Summary 54
Ability Manager 55
1. Overview 55
2. Main Features 55
3. Data Structure 55
abilities 55
slots 55
4. API Reference 55

Table of Content

4.1 AbilityManager.new() 55
4.2 AbilityManager.add(manager, ability) 56
4.3 AbilityManager.get(manager, name) 56
4.4 AbilityManager.has(manager, name) 56
4.5 AbilityManager.remove(manager, name) 57
4.6 AbilityManager.update(manager, dt) 57
4.7 AbilityManager.assign_slot(manager, slot, ability_name) 58
4.8 AbilityManager.get_slot(manager, slot) 58
4.9 AbilityManager.clear_slot(manager, slot) 58
4.10 AbilityManager.use_slot(manager, slot, data) 59
5. Typical Usage Example 59
6. Implementation Notes 60
6.1 Relationship with the Ability System 60
6.2 Slot Assignments Are Name-Based 60
6.3 Multiple Slots Can Reference the Same Ability 60
6.4 Slot Identifiers 61
6.5 Shared Versus Independent State 61
7. Limitations 61
8. Quick Reference 61
9. Usage Guidelines 61
10. Summary 62

Checkpoint System 63
1. Overview 63
2. Main Features 63
3. Data Structure 63
4. API Reference 63
4.1 Checkpoint.new() 63
4.2 Checkpoint.set(checkpoint, room_name, position) 63
4.3 Checkpoint.get_room(checkpoint) 64
4.4 Checkpoint.get_position(checkpoint) 64
4.5 Checkpoint.clear(checkpoint) 65
5. Typical Usage Example 65
6. Implementation Notes 65
6.1 Data Storage 65
6.2 Position Representation 66
6.3 Room Validation 66
6.4 Validation Scope 66
7. Limitations 66
8. Quick Reference 66
9. Usage Guidelines 66
10. Summary 67

Health System 68
1. Overview 68

Table of Content

2. Main Features 68
3. Data Structure 68
4. API Reference 68
4.1 `Health.new(max_health)` 68
4.2 `Health.damage(health, amount)` 69
4.3 `Health.heal(health, amount)` 69
4.4 `Health.get_current(health)` 70
4.5 `Health.get_max(health)` 70
4.6 `Health.is_dead(health)` 70
4.7 `Health.revive(health, amount)` 71
5. Typical Usage Example 71
6. Implementation Notes 72
6.1 Independent Instances 72
6.2 Damage and Healing Limits 72
6.3 Death State 72
6.4 Revival Semantics 72
6.5 Validation Scope 72
7. Limitations 72
8. Quick Reference 73
9. Usage Guidelines 73
10. Summary 73
Inventory System 74
1. Overview 74
2. Main Features 74
3. Data Structure 74
4. API Reference 74
4.1 `Inventory.new()` 74
4.2 `Inventory.add(inventory, item_name, amount)` 75
4.3 `Inventory.remove(inventory, item_name, amount)` 75
4.4 `Inventory.has(inventory, item_name)` 76
4.5 `Inventory.get_count(inventory, item_name)` 76
4.6 `Inventory.clear(inventory)` 77
4.7 `Inventory.get_items(inventory)` 77
5. Typical Usage Example 78
6. Implementation Notes 78
6.1 Item Identification 78
6.2 Quantity Management 78
6.3 No Item Objects 79
6.4 No Capacity Limit 79
6.5 Validation Scope 79
7. Limitations 79
8. Quick Reference 79
9. Usage Guidelines 80
10. Summary 80

Table of Content

Respawn System **81**
1. Overview 81
2. Main Features 81
3. Dependency 81
4. API Reference 81
4.1 `Respawn.get_room(checkpoint)` 81
4.2 `Respawn.get_position(checkpoint)` 82
4.3 `Respawn.can_respawn(checkpoint)` 82
5. Typical Usage Example 82
6. Implementation Notes 83
6.1 No Independent State 83
6.2 Delegated Data Access 83
6.3 Respawn Eligibility 83
6.4 Error Handling 83
7. Limitations 83
8. Quick Reference 84
9. Usage Guidelines 84
10. Summary 84
Room Manager **85**
1. Overview 85
2. Features 85
3. Data Structure 85
4. API Reference 85
4.1. `RoomManager.new()` 85
4.2. `RoomManager.enter(manager, room_name)` 85
4.3. `RoomManager.get_current(manager)` 86
4.4. `RoomManager.is_current(manager, room_name)` 86
4.5. `RoomManager.clear(manager)` 87
5. Usage Example 87
6. Implementation Notes 87
7. Limitations 88
8. Quick Reference 88
9. Usage Guidelines 88
10. Summary 88
Stats System **89**
1. Overview 89
2. Features 89
3. Data Structure 89
4. API Reference 89
4.1. `Stats.new()` 89
4.2. `Stats.set(stats, name, value)` 90
4.3. `Stats.get(stats, name)` 90
4.4. `Stats.modify(stats, name, amount)` 90
4.5. `Stats.has(stats, name)` 91
4.6. `Stats.set_max(stats, name, max_value)` 91

4.7. `Stats.get_max(stats, name)` 92
4.8. `Stats.set_min(stats, name, min_value)` 92
4.9. `Stats.get_min(stats, name)` 92
4.10. `Stats.remove(stats, name)` 93
5. Usage Example 93
6. Implementation Notes 94
7. Limitations 94
8. Quick Reference 94
9. Usage Guidelines 94
10. Summary 95
Gameplay Object System 96
1. Overview 96
2. Features 96
3. Data Structure 96
4. API Reference 96
4.1. `GameplayObject.new()` 96
4.2. `GameplayObject.add(gameplay_object, name, component)` 96
4.3. `GameplayObject.get(gameplay_object, name)` 97
4.4. `GameplayObject.has(gameplay_object, name)` 97
4.5. `GameplayObject.remove(gameplay_object, name)` 98
4.6. `GameplayObject.clear(gameplay_object)` 98
5. Usage Example 98
6. Implementation Notes 99
7. Limitations 99
8. Quick Reference 99
9. Usage Guidelines 100
10. Summary 100

Event Bus

File: `scripts/core/event_bus.lua`
Category: Core Systems
Version: 1.0
Status: Implemented

1. Overview

The Event Bus is a centralized event communication system that allows different parts of an application to communicate without requiring direct references to one another.

It supports event subscription, event emission, subscription removal, one-time listeners, and listener management.

Key Features

- Event subscription and emission.
- Multiple listeners per event.
- Duplicate subscription prevention.
- One-time event listeners.
- Individual and global listener removal.
- Listener existence checks.
- Snapshot-based listener iteration to protect the current iteration from modifications to the original listener list.

Design Principles

The Event Bus uses a publish-subscribe pattern.

A system can emit an event without knowing which systems are listening for it. Likewise, a listener does not need to know which system emitted the event.

For example, when a player collects an item, the inventory system could emit an event that the UI and audio systems listen for independently.

The Event Bus itself does not implement these gameplay integrations; they are examples of how the system can be used.

2. API Reference

The module exposes seven public functions.

Function	Description
`on(event_name, callback)`	Registers an event listener.
`emit(event_name, data)`	Emits an event and notifies its listeners.
`off(event_name, callback)`	Removes a specific listener.
`once(event_name, callback)`	Registers a listener that executes once.
`clear(event_name)`	Removes all listeners for a specific event.
`clear_all()`	Removes all registered listeners.
`has(event_name)`	Checks whether an event has any listeners.

2.1. `EventBus.on(event_name, callback)`

Description

Registers a callback function to listen for a specific event.

If the event does not have a listener list, one is created automatically.

Registering the same callback for the same event more than once is prevented.

Parameters

Name	Type	Description
`event_name`	`string`	The name of the event to subscribe to.
`callback`	`function`	The function to execute when the event is emitted.

Returns

`function` — An unsubscribe function that removes the registered listener.

If the same callback is already registered for the same event, the function returns a no-op function instead. Calling that function does not remove the existing subscription.

Example:

```
local EventBus = require "scripts.core.event_bus"


local function on_player_damaged(data)
  print("Damage received:", data.amount)
end


local unsubscribe = EventBus.on(
  "player_damaged",
  on_player_damaged
)

EventBus.emit("player_damaged", {
  amount = 25
})


-- Remove the subscription.
unsubscribe()
```

Behavior

- Creates an event listener list when necessary.
- Prevents duplicate registrations of the same callback for the same event.
- Allows the same callback to subscribe to different events.
- Returns a function for convenient unsubscription.

Validation

The function asserts that:

- `event_name` is a string.
- `callback` is a function.

Important Notes

Duplicate detection is based on function identity, not function contents. Two separate functions containing identical code are considered different callbacks.

2.2. `EventBus.emit(event_name, data)`

Description

Emits an event and passes the supplied data to every listener registered for that event.

Parameters

Name	Type	Description
`event_name`	`string`	The name of the event to emit.
`data`	Any Lua value	The data passed to each listener.

The `data` parameter is optional in practice. If omitted, the callback receives `nil`.

Returns

No explicit return value.

Example:

```
EventBus.emit("player_damaged", {
    amount = 25,
    source = "enemy"
})
```

A listener can access the supplied data:

```
local function on_player_damaged(data)
    print("Damage:", data.amount)
    print("Source:", data.source)
end
```

Behavior

1. Retrieves the listener list for the specified event.
2. Returns immediately if no listeners exist.
3. Creates a snapshot of the current listener list.
4. Iterates over the snapshot and invokes each callback, passing `data`.

Snapshot Iteration

The snapshot prevents modifications to the original listener list from directly changing the list currently being iterated over.

This provides the following behavior:

- A listener added during emission is not called during that same emission.
- Removing a listener during emission does not remove it from the existing snapshot.
- A listener that was already present in the snapshot may still be called during the current emission even if its subscription is removed before its turn.

Subsequent emissions use the updated listener list.

Error Handling

The function does not use `pcall` or another error-handling mechanism around listener execution.

If a callback raises an error, the error propagates to the caller, and the remaining callbacks in that emission are not executed.

Validation

The function does not explicitly validate `event_name` or `data`.

2.3. `EventBus.off(event_name, callback)`

Description

Removes a specific callback from an event's listener list.

Parameters

Name	Type	Description
`event_name`	`string`	The event from which to remove the listener.
`callback`	`function`	The callback to remove.

Returns

No explicit return value.

Example

```
local function on_player_damaged(data)
    print("Player damaged")
end

EventBus.on("player_damaged", on_player_damaged)

EventBus.off("player_damaged", on_player_damaged)
```

Behavior

- Does nothing if the event has no listener list.
- Searches for the specified callback.
- Removes the first matching callback.
- Deletes the event's listener list if it becomes empty.

Because `on()` prevents duplicate registrations of the same callback for the same event, there should normally be at most one matching entry.

Important Notes

The function does not explicitly validate its arguments. Passing an invalid event name or callback may produce unexpected behavior.

2.4. `EventBus.once(event_name, callback)`

Description

Registers a callback that executes only once for a particular event subscription.

After the event is received, the internal listener automatically unsubscribes before invoking the supplied callback.

Parameters

Name	Type	Description
`event_name`	`string`	The event to listen for.
`callback`	`function`	The callback to execute once.

Returns

`function` — An unsubscribe function that can be called to cancel the subscription before it executes.

Example

```
EventBus.once("boss_defeated", function(data)
    print("Boss defeated:", data.name)
end)
```

The callback will execute on the first matching event emission and will not execute again through that subscription.

Implementation Details

The function internally uses `EventBus.on()` to register a wrapper callback.

When the wrapper receives the event, it:

1. Unsubscribes itself.
2. Invokes the original callback with the received data.

Unsubscribing before invoking the original callback prevents the same subscription from executing again through a nested emission triggered by that callback.

Important Notes

Because `once()` uses `on()`, it inherits the duplicate-subscription behavior of `on()`.

If the exact wrapper callback were already registered, `on()` would return a no-op unsubscribe function. In ordinary use, the wrapper is newly created for each call, so this does not normally arise.

2.5. `EventBus.clear(event_name)`

Description

Removes all listeners registered for a specific event.

Parameters

Name	Type	Description
`event_name`	`string`	The event whose listeners should be removed.

Returns

No explicit return value.

Example

```
EventBus.clear("player_damaged")
```

After this call, subsequent emissions of `"player_damaged"` will not invoke the listeners previously registered for that event.

Listeners belonging to other events remain unaffected.

Important Notes

The function removes the event's listener list by setting its entry to `nil`.

If called during an ongoing emission, it does not invalidate the snapshot already created by that emission. Listeners present in that snapshot may still execute during the current emission.

2.6. `EventBus.clear_all()`

Description

Removes all registered event listeners across the entire Event Bus.

Parameters

None.

Returns

No explicit return value.

Example

```
EventBus.clear_all()
```

Behavior

Replaces the internal listener table with a new empty table.

All subsequent event emissions will have no listeners until new subscriptions are registered.

Important Notes

This function has global effects because the Event Bus maintains shared module-level state.

It should not be used when only one listener or one event needs to be cleared.

Like `clear()`, it does not modify snapshots already created by an emission currently in progress.

2.7. `EventBus.has(event_name)`

Description

Checks whether an event currently has at least one registered listener.

Parameters

Name	Type	Description
`event_name`	`string`	The event to check.

Returns

`boolean`

- `true` if the event has one or more registered listeners.
- `false` otherwise.

Example

```
if EventBus.has("boss_defeated") then
   print("The event has listeners")
end
```

Behavior

The function returns `true` only if a listener list exists and contains at least one entry.

It does not execute the listeners or inspect their internal behavior.

3. Internal Implementation

3.1. Listener Storage

The module maintains a local table:

```
local listeners = {}
```

Each event name maps to an array of callback functions.

Conceptually, the structure looks like this:

```
listeners
├── "player_damaged"
|   ├── callback_1
|   └── callback_2
|
├── "boss_defeated"
|   └── callback_3
|
└── "item_collected"
    ├── callback_4
    └── callback_5
```

The actual event names and callback functions depend on runtime subscriptions.

3.2. Shared Module State

The Event Bus does not expose a constructor such as `EventBus.new()`.

Instead, its listener table is shared by all callers that use the same loaded module instance.

This means that one script can register an event listener while another script emits the event through the same module.

It also means that `clear_all()` affects all subscriptions managed by that module instance, not merely those belonging to a particular gameplay object.

3.3. Event Dispatch

Event dispatch is synchronous.

When `emit()` is called, listeners are invoked during that call. The Event Bus does not queue events for later processing or schedule callbacks on another frame.

4. Limitations

The current implementation has the following limitations:

Limitation	Description
No priority system	Listeners execute in registration order.
No asynchronous dispatch	Listeners execute synchronously during `emit()`.
No built-in error isolation	An error in one listener can prevent subsequent listeners from executing.
No event registry	Event names are arbitrary strings rather than predefined identifiers.
No return-value aggregation	Results returned by listeners are ignored.
No automatic lifecycle management	Callers must remove subscriptions when they are no longer needed.

Limited argument validation	Only `on()` explicitly validates both of its parameters.
Shared global state	All callers using the same module instance share the same subscriptions.

These are limitations of the current implementation, not necessarily requirements for future development.

They should not be treated as reasons to expand the system automatically. Any proposed enhancement should solve a demonstrated need.

5. Quick Reference

Operation	Usage
Subscribe to an event	`EventBus.on("event_name", callback)`
Emit an event	`EventBus.emit("event_name", data)`
Unsubscribe a callback	`EventBus.off("event_name", callback)`
Subscribe once	`EventBus.once("event_name", callback)`
Clear one event	`EventBus.clear("event_name")`
Clear all events	`EventBus.clear_all()`
Check for listeners	`EventBus.has("event_name")`

6. Usage Guidelines

- Use descriptive event names, such as `"player_damaged"` or `"item_collected"`.
- Prefer passing a table when an event requires multiple pieces of data.
- Keep callbacks focused on responding to the event rather than implementing unrelated logic.
- Store the unsubscribe function returned by `on()` when a listener must later be removed.
- Use `once()` for one-time reactions.
- Avoid calling `clear_all()` unless clearing every subscription is intentional.
- Remove subscriptions when their owning objects or scripts no longer need them.

Example of a table-based event payload:

```
EventBus.emit("item_collected", {
  item_id = "health_shard",
  amount = 1
})
```

The payload structure is determined by the code that emits the event. The Event Bus does not validate or enforce a particular schema.

7. Summary

The Event Bus provides a lightweight, module-level publish-subscribe mechanism for communication between systems.

Its current implementation supports:

- Registration and removal of listeners.
- Synchronous event emission.
- Optional event payloads.
- One-time subscriptions.
- Duplicate callback prevention.
- Snapshot-based iteration.
- Individual and global listener management.

It is implemented entirely in Lua and does not require a native extension.

Document status: `Draft 1.0` — based on the supplied implementation of `scripts/core/event_bus.lua`.

Timer

File: `scripts/core/timer.lua`
Category: Core Systems
Version: 1.0
Status: Implemented

1. Overview

The Timer module provides a simplified interface for scheduling delayed and repeating callbacks using Defold's built-in timer API.

It supports one-shot timers, repeating timers, individual timer cancellation, and cancellation of all timers managed by the module.

The module delegates timer execution to Defold through `timer.delay()` and `timer.cancel()`. It does not implement its own timing mechanism.

Key Features

- Execute a callback after a specified delay.
- Execute a callback repeatedly at a specified interval.
- Pass optional data to callbacks.
- Cancel an individual timer.
- Cancel all timers tracked by the module.
- Track timer identifiers created through the module.

Design Principles

The Timer module is a lightweight wrapper around Defold's existing timer functionality.

Its purpose is to simplify common timer operations and provide centralized tracking of timers created through this module, rather than replacing Defold's native timer system.

2. API Reference

The module exposes four public functions.

Function	Description
`after(delay, callback, data)`	Schedules a callback to execute once after a delay.
`every(interval, callback, data)`	Schedules a callback to execute repeatedly.
`cancel(timer_id)`	Cancels a timer and removes its identifier from the module's tracking table.
`cancel_all()`	Cancels all timers currently tracked by the module.

2.1. `Timer.after(delay, callback, data)`

Description

Schedules a callback to execute once after the specified delay.

The function uses Defold's `timer.delay()` with its repetition parameter set to `false`.

After the callback executes successfully, the timer is cancelled through `Timer.cancel()` to remove its identifier from the module's tracking table.

Parameters

Name	Type	Description
`delay`	`number`	The delay before the callback executes, in seconds.
`callback`	`function`	The function to execute after the delay.
`data`	Any Lua value	Optional data passed to the callback.

Returns

`timer_id` — The timer identifier returned by Defold's `timer.delay()`.

The identifier can be passed to `Timer.cancel()` to cancel the timer before it executes.

Example

```
local Timer = require "scripts.core.timer"

Timer.after(2, function()
    print("Two seconds have passed.")
end)
```

The callback is scheduled to execute after approximately two seconds.

Example with Data

```
local Timer = require "scripts.core.timer"

Timer.after(3, function(data)
    print("Message:", data.message)
end, {
    message = "Delayed callback executed."
})
```

The supplied data is passed to the callback when the timer fires.

Behavior

1. Validates the delay and callback types.
2. Creates a one-shot Defold timer.
3. Stores the returned timer identifier in the module's tracking table.
4. Executes the callback when the timer fires.
5. Calls `Timer.cancel()` to remove the timer identifier from the tracking table.

Validation

The function asserts that:

- `delay` is a number.
- `callback` is a function.

It does not check whether the delay is positive or non-negative.

Important Notes

The timer identifier is tracked until the callback executes successfully or the timer is cancelled through the module.

If the callback raises an error, execution stops before `Timer.cancel(timer_id)` is reached. The timer identifier may therefore remain in the tracking table even though Defold's one-shot timer has already fired.

The function does not catch callback errors.

2.2. `Timer.every(interval, callback, data)`

Description

Schedules a callback to execute repeatedly at the specified interval.

The function uses Defold's `timer.delay()` with its repetition parameter set to `true`.

The timer continues repeating until it is cancelled.

Parameters

Name	Type	Description
`interval`	`number`	The repeating interval, in seconds.
`callback`	`function`	The function to execute each time the timer fires.
`data`	Any Lua value	Optional data passed to every callback invocation.

Returns

`timer_id` — The timer identifier returned by Defold's `timer.delay()`.

The identifier can be passed to `Timer.cancel()` to stop the repeating timer.

Example

```
local Timer = require "scripts.core.timer"

Timer.every(1, function()
    print("One second elapsed.")
end)
```

The callback is scheduled to execute repeatedly at one-second intervals.

Example with Data

```
local Timer = require "scripts.core.timer"

Timer.every(2, function(data)
    print(data.message)
end, {
    message = "Repeating timer executed."
})
```

The same data value is passed to the callback on every invocation.

Behavior

1. Validates the interval and callback types.
2. Creates a repeating Defold timer.
3. Stores the returned timer identifier in the module's tracking table.
4. Executes the callback whenever the timer fires.
5. Continues repeating until cancelled.

Validation

The function asserts that:

- `interval` is a number.
- `callback` is a function.

It does not check whether the interval is positive or non-negative.

Important Notes

Unlike `Timer.after()`, this function does not automatically cancel the timer after a callback executes.

The caller is responsible for cancelling the timer when it is no longer needed.

2.3. `Timer.cancel(timer_id)`

Description

Cancels a timer through Defold's native timer API and removes the first matching identifier from the module's tracking table.

Parameters

Name	Type	Description
`timer_id`	Defold timer identifier	The identifier of the timer to cancel.

Returns

No explicit return value.

Example

```
local Timer = require "scripts.core.timer"

local timer_id = Timer.every(1, function()
    print("Repeating...")
end)

-- Cancel the repeating timer.
Timer.cancel(timer_id)
```

Behavior

1. Calls `timer.cancel(timer_id)`.
2. Searches the module's tracking table for the supplied identifier.
3. Removes the first matching entry if one exists.

Important Notes

The function calls Defold's `timer.cancel()` even if the identifier is not present in the module's tracking table.

It does not explicitly validate the identifier or verify that the timer is still active.

Calling this function with an invalid or already-cancelled identifier is not handled specially by the module.

2.4. `Timer.cancel_all()`

Description

Cancels every timer identifier currently stored in the module's tracking table.

This function is useful when a system needs to stop all timers it has created through this module.

Parameters

None.

Returns

No explicit return value.

Example

```
local Timer = require "scripts.core.timer"

Timer.cancel_all()
```

Behavior

1. Iterates over the tracked timer identifiers.
2. Calls Defold's `timer.cancel()` for each identifier.
3. Replaces the tracking table with a new empty table.

Important Notes

This function affects all timers tracked by the shared Timer module instance, regardless of which script created them.

It does not cancel timers created directly through Defold's `timer.delay()` API outside this module, because those timers are not automatically registered in the module's tracking table.

Use this function only when cancelling every timer managed by the module is intentional.

3. Internal Implementation

3.1. Timer Tracking

The module maintains a local table:

```
local timers = {}
```

Every timer created through `Timer.after()` or `Timer.every()` has its identifier added to this table.

Conceptually, the table may contain entries such as:

```
timers
├── timer_id_1
├── timer_id_2
└── timer_id_3
```

These identifiers allow the module to cancel its tracked timers collectively.

3.2. Integration with Defold

The module delegates timer creation and cancellation to Defold:

- `timer.delay()` creates one-shot or repeating timers.
- `timer.cancel()` cancels timers.

The module does not implement time measurement, frame updates, or callback scheduling itself.

3.3. Shared Module State

The Timer module does not expose a constructor such as `Timer.new()`.

Its tracking table is shared by callers using the same loaded module instance.

Consequently, `Timer.cancel_all()` operates on the combined set of timers tracked through that instance rather than on a separate collection belonging to an individual caller.

3.4. Timer Lifecycle

One-shot timer lifecycle

```
Timer.after()
      |
      v
Create Defold timer
      |
      v
Store timer identifier
      |
      v
Wait for the delay
      |
      v
Execute callback
      |
      v
Timer.cancel()
      |
      v
Remove identifier from tracking table
```

If the callback raises an error, execution does not reach the final cancellation step.

Repeating timer lifecycle

```
Timer.every()
      |
      v
Create repeating Defold timer
      |
```

```
      v
Store timer identifier
      |
      v
Execute callback repeatedly
      |
      v
Timer.cancel(timer_id)
      |
      v
Remove identifier from tracking table
```

The repeating timer remains active until it is cancelled.

4. Limitations

The current implementation has the following limitations:

Limitation	Description
No delay or interval range validation	Numeric values are accepted without checking whether they are positive or non-negative.
No callback error isolation	An error inside a callback propagates to the caller.
Potential stale tracking entry	If an `after()` callback raises an error, its identifier may remain in the tracking table.
No automatic lifecycle cleanup	Timers must be cancelled when they are no longer needed.
Shared timer tracking	`cancel_all()` affects all timers tracked by the shared module instance.
No ownership tracking	The module does not associate timers with individual gameplay objects or systems.
No completion callback	The module does not provide a separate notification indicating that a timer has completed.
No direct timer-state query	The module does not expose a function to check whether a timer is active.
No protection against invalid identifiers	The module delegates cancellation directly to Defold without additional checks.

These limitations describe the current implementation. They do not automatically justify adding more functionality.

Any future extension should address a concrete requirement rather than duplicate capabilities already provided by Defold.

5. Quick Reference

Operation	Usage
Schedule a one-shot callback	`Timer.after(2, callback)`
Schedule a repeating callback	`Timer.every(1, callback)`
Pass data to a one-shot callback	`Timer.after(2, callback, data)`
Pass data to a repeating callback	`Timer.every(1, callback, data)`
Cancel one timer	`Timer.cancel(timer_id)`
Cancel all tracked timers	`Timer.cancel_all()`

All delays and intervals are expressed in seconds.

6. Usage Guidelines

- Use `Timer.after()` for delayed actions that should occur once.
- Use `Timer.every()` for actions that must repeat until explicitly cancelled.
- Store the returned timer identifier when individual cancellation may be required.
- Pass optional callback data instead of relying unnecessarily on external mutable variables.
- Cancel repeating timers when their associated behavior is no longer needed.
- Avoid calling `Timer.cancel_all()` from a system that should only cancel its own timers.

- Prefer this module when centralized timer tracking is useful; use Defold's native timer API directly when the wrapper provides no benefit.
- Keep callback logic focused and account for the fact that callback errors are not caught by this module.

7. Summary

The Timer module is a lightweight wrapper around Defold's native timer functionality.

It provides:

- One-shot delayed callbacks.
- Repeating callbacks.
- Optional callback data.
- Individual timer cancellation.
- Cancellation of all tracked timers.

It also maintains a shared list of timer identifiers created through the module.

The implementation does not replace Defold's timer system. It builds a small convenience layer on top of it.

Document status: `Draft 1.0` — based on the supplied implementation of `scripts/core/timer.lua`.

State Machine

File: `scripts/core/state_machine.lua`
Category: Core Systems
Version: 1.0
Status: Implemented

1. Overview

The State Machine module provides a lightweight mechanism for managing named states and transitioning between them.

Each state can define three optional callbacks:

- `on_enter`: Executed when the state becomes active.
- `on_exit`: Executed when the state is about to be replaced by another state.
- `on_update`: Executed when the active state is updated.

The module tracks the current state and provides functions to register, remove, inspect, update, and transition between states.

Key Features

- Create independent state machines.
- Register named states.
- Transition between registered states.
- Execute callbacks when entering or exiting states.
- Update the active state.
- Retrieve the current state.
- Check whether a state exists or is currently active.
- Remove states that are not currently active.

Design Principles

The State Machine is designed to manage state-related logic without requiring all state behavior to be placed in one large function.

For example, a character might use states such as:

- `idle`
- `running`
- `jumping`
- `attacking`
- `dead`

These names are illustrative; the module does not define or enforce any particular set of states.

The State Machine does not implement character movement, animation, combat, or transition rules. Those responsibilities belong to the systems that use it.

2. API Reference

The module exposes seven public functions.

Function	Description
`new(initial_state)`	Creates a new state machine.
`add(machine, name, on_enter, on_exit, on_update)`	Registers a state and its optional callbacks.
`set(machine, new_state)`	Transitions to a registered state.
`get(machine)`	Returns the current state name.
`remove(machine, name)`	Removes a state that is not currently active.
`has(machine, name)`	Checks whether a state is registered.

`update(machine, dt)`	Executes the active state's update callback, if available.
`is(machine, state_name)`	Checks whether the specified state is currently active.

2.1. `StateMachine.new(initial_state)`

Description

Creates and returns a new state machine.

The machine stores the supplied initial state name and initializes an empty table for registered states.

Parameters

Name	Type	Description
`initial_state`	Any non-`nil` Lua value	The initial current state value. In normal usage, this should be a string representing a state name.

Returns

A new state machine table with the following structure:

```
{
    current = initial_state,
    states = {}
}
```

Example

```
local StateMachine = require
"scripts.core.state_machine"
local machine = StateMachine.new("idle")
```

Important Notes

The initial state does not need to be registered before the machine is created.

However, the machine will not have any callbacks associated with that state until it is registered through `StateMachine.add()`.

The constructor only checks that `initial_state` is not `nil`. It does not validate that the value is a string.

2.2. `StateMachine.add(machine, name, on_enter, on_exit, on_update)`

Description

Registers a state in the specified state machine.

Each state is stored by name and may define callbacks for entering, exiting, and updating that state.

Parameters

Name	Type	Description
`machine`	State machine table	The machine that will contain the state.
`name`	`string`	The name of the state.
`on_enter`	`function` or `nil`	Optional callback executed when the state becomes active.
`on_exit`	`function` or `nil`	Optional callback executed when leaving the state.
`on_update`	`function` or `nil`	Optional callback executed when the active state is updated.

All three callbacks are optional.

Returns

No explicit return value.

Example

```
StateMachine.add(
    machine,
    "idle",
    function(machine, state_name)
        print("Entered:", state_name)
    end,
    function(machine, state_name)
        print("Exited:", state_name)
    end,
    function(machine, dt)
        -- Update idle-state behavior.
    end
)
```

Callback Signatures

Callback	Arguments
`on_enter`	`(machine, new_state)`
`on_exit`	`(machine, old_state)`
`on_update`	`(machine, dt)`

The `machine` argument is the state machine table itself.

The `dt` argument is passed through from the caller of `StateMachine.update()`.

Behavior

The function stores the callbacks in a table associated with the supplied state name.

If a state with the same name already exists, its previous callback table is replaced.

Validation

The function asserts that:

- `machine` is not `nil`.
- `name` is a string.
- `on_enter` is either `nil` or a function.
- `on_exit` is either `nil` or a function.
- `on_update` is either `nil` or a function.

Important Notes

Registering a state does not automatically activate it or execute its `on_enter` callback.

If the registered state is already the current state, its callbacks become available for subsequent operations, but `on_enter` is not called by `add()`.

2.3. `StateMachine.set(machine, new_state)`

Description

Transitions the machine from its current state to a different registered state.

The function handles the old state's exit callback, updates the current state, and then invokes the new state's entry callback.

Parameters

Name	Type	Description
`machine`	State machine table	The machine to modify.
`new_state`	Any non-`nil` Lua value	The name of the registered state to activate. In normal usage, this should be a string.

Returns

The return value depends on whether a transition occurs.

Situation	Return value
Transition succeeds	`old_state, new_state`
Requested state is already active	`false`

If the requested state is not registered, the function raises an assertion error.

Example

```
StateMachine.add(machine, "idle")
StateMachine.add(machine, "running")

local old_state, new_state = StateMachine.set(
    machine,
    "running"
)

print(old_state) -- idle
print(new_state) -- running
```

Transition Sequence

When transitioning to a different registered state, the function performs these operations in order:

1. Retrieves the target state's callback table.
2. Retrieves the current state's callback table, if registered.
3. Executes the current state's `on_exit` callback, if available.
4. Updates `machine.current` to the new state.
5. Executes the new state's `on_enter` callback, if available.
6. Returns the old and new state names.

Transitioning to the Current State

If `machine.current` already equals `new_state`, the function immediately returns `false`.

No callbacks are executed, and the current state remains unchanged.

Validation

The function asserts that:

- `machine` is not `nil`.
- `new_state` is not `nil`.
- The requested state is registered, unless it is already the current state.

Important Notes

The target state's existence is checked before the old state's exit callback executes. An invalid transition therefore raises an error without first invoking `on_exit`.

The function does not automatically manage transitions based on input, timers, or gameplay conditions. The caller must decide when to request a transition.

Callbacks are executed synchronously. If an `on_exit` callback raises an error, the assignment to `machine.current` is not reached. If an `on_enter` callback raises an error, the current state has already been changed.

2.4. `StateMachine.get(machine)`

Description

Returns the machine's current state value.

Parameters

Name	Type	Description
`machine`	State machine table	The machine to inspect.

Returns

The value stored in `machine.current`.

Example

```
local current_state = StateMachine.get(machine)

print("Current state:", current_state)
```

Validation

The function asserts that `machine` is not `nil`.

It does not verify that the current state is registered.

2.5. `StateMachine.remove(machine, name)`

Description

Removes a registered state from the machine.

The currently active state cannot be removed.

Parameters

Name	Type	Description
`machine`	State machine table	The machine containing the state.
`name`	`string`	The name of the state to remove.

Returns

No explicit return value.

Example

```
StateMachine.add(machine, "temporary_state")

StateMachine.remove(machine, "temporary_state")
```

Behavior

The function sets the specified entry in `machine.states` to `nil`.

Validation

The function asserts that:

- `machine` is not `nil`.
- `name` is a string.
- `name` is not equal to the current state.

Important Notes

The function does not check whether the specified state actually exists.

Attempting to remove a nonexistent state simply leaves the state table unchanged.

Removing the current state raises an assertion error.

2.6. `StateMachine.has(machine, name)`

Description

Checks whether a state is registered in the machine.

Parameters

Name	Type	Description
`machine`	State machine table	The machine to inspect.
`name`	`string`	The state name to check.

Returns

`boolean`

- `true` if the state is registered.
- `false` otherwise.

Example

```
if StateMachine.has(machine, "running") then
    print("The running state is registered.")
end
```

Validation

The function asserts that:

- `machine` is not `nil`.
- `name` is a string.

Important Notes

This function checks registration, not activation.

A registered state may exist without being the current state.

2.7. `StateMachine.update(machine, dt)`

Description

Executes the update callback of the currently active state, if the state is registered and defines an `on_update` callback.

Parameters

Name	Type	Description
`machine`	State machine table	The machine to update.
`dt`	`number` in typical usage	The delta time supplied by the caller.

Returns

No explicit return value.

Example

```
function update(self, dt)
    StateMachine.update(self.state_machine, dt)
end
```

This example assumes that `self.state_machine` has already been initialized.

Behavior

1. Retrieves the callback table associated with the current state.
2. Checks whether that state exists and defines an `on_update` callback.
3. Executes the callback with `(machine, dt)` if available.

Validation

The function asserts that `machine` is not `nil`.

It does not validate the type or value of `dt`.

Important Notes

If the current state has not been registered, the function does nothing.

Likewise, if the current state has no `on_update` callback, no action is performed.

The module does not call `StateMachine.update()` automatically. The caller must invoke it at the appropriate time, such as during a Defold script's `update()` callback.

2.8. `StateMachine.is(machine, state_name)`

Description

Checks whether the specified state is currently active.

Parameters

Name	Type	Description
`machine`	State machine table	The machine to inspect.
`state_name`	`string`	The state name to compare against the current state.

Returns

`boolean`

- `true` if `machine.current` equals `state_name`.
- `false` otherwise.

Example

```
if StateMachine.is(machine, "running") then
    print("The machine is currently running.")
end
```

Validation

The function asserts that:

- `machine` is not `nil`.
- `state_name` is a string.

Important Notes

The function compares the requested name directly with `machine.current`. It does not require the state to be registered.

3. Internal Implementation

3.1. State Machine Structure

Each state machine is represented by a Lua table containing two fields:

```
    {
        current = "idle",

        states = {
            idle = {
                on_enter = nil,
                on_exit = nil,
                on_update = nil
            }
```

```
        }
    }
```

This is a conceptual representation. The actual callback values depend on the states registered by the caller.

3.2. Independent Instances

Unlike the Event Bus and Timer modules, the State Machine module does not store all machines in one shared state table.

Each call to `StateMachine.new()` returns a separate machine table with its own:

- Current state.
- Registered states.
- State callbacks.

Multiple state machines can therefore operate independently while using the same module.

3.3. State Registration and Activation

Registering a state and activating a state are separate operations.

`StateMachine.add()` makes a state available for transitions.
`StateMachine.set()` changes the active state.

The constructor initializes `current` directly, but does not automatically register the initial state.

Consequently, the initial state may temporarily be unregistered. In that situation, `update()` does nothing until a matching state is registered, and the old state's exit callback is unavailable if a transition occurs before registration.

3.4. State Transition Lifecycle

A successful transition follows this sequence:

```
StateMachine.set(machine, new_state)
                  |
                  v
       Is the state different?
                  |
                  v
       Is the target registered?
                  |
                  v
        Execute old on_exit
                  |
                  v
       Update machine.current
                  |
                  v
        Execute new on_enter
                  |
                  v
       Return old and new states
```

If the target is already active, the function returns `false` without executing callbacks.

If the target is not registered, the function raises an error before executing the old state's exit callback.

4. Limitations

The current implementation has the following limitations:

Limitation	Description
No automatic transition rules	Callers must decide when to transition between states.
No initial-state registration	`new()` initializes the current state but does not register its callbacks.

No transition validation beyond registration	Any registered state can be selected; allowed transition paths are not defined.
No nested state machines	The module does not provide built-in hierarchical or nested state support.
No transition priorities	The module does not resolve competing transition requests.
No callback error isolation	Errors in callbacks propagate to the caller.
No automatic update scheduling	The caller must invoke `update()` manually.
No transition history	Previous states are not stored automatically.
No automatic cleanup	State callbacks are removed only when the caller removes their state or discards the machine.

These limitations reflect the current scope of the module. Additional features should be considered only when actual gameplay systems require them.

5. Quick Reference

Operation	Usage
Create a state machine	`StateMachine.new("idle")`
Register a state	`StateMachine.add(machine, "running")`
Register callbacks	`StateMachine.add(machine, name, on_enter, on_exit, on_update)`
Change the current state	`StateMachine.set(machine, "running")`
Get the current state	`StateMachine.get(machine)`
Remove a state	`StateMachine.remove(machine, "running")`
Check whether a state exists	`StateMachine.has(machine, "running")`
Update the active state	`StateMachine.update(machine, dt)`
Check the active state	`StateMachine.is(machine, "running")`

6. Usage Guidelines

- Register the states required by a machine before relying on their callbacks.
- Use `on_enter` for initialization associated with entering a state.
- Use `on_exit` for cleanup associated with leaving a state.
- Use `on_update` for logic that must run while a state is active.
- Call `StateMachine.update()` from the appropriate update loop.
- Use `StateMachine.is()` when behavior depends on the currently active state.
- Keep transition decisions in the system that owns the gameplay rules.
- Avoid placing unrelated gameplay systems inside state callbacks.
- Remember that callbacks receive the machine itself, allowing them to inspect or modify its state through the public API.
- Do not remove the active state; transition to another registered state first.

7. Summary

The State Machine module provides a simple, reusable foundation for managing named states and their associated callbacks.

It supports:

- Independent state machine instances.
- State registration and removal.
- State transitions.
- Entry, exit, and update callbacks.
- Current-state inspection.
- State registration checks.
- Active-state checks.

The module does not define gameplay states or transition rules. It provides the mechanism, while the systems that use it determine the meaning of each state and when transitions should occur.

Document status: `Draft 1.0` — based on the supplied implementation of `scripts/core/state_machine.lua`.

Input Bindings

File: `scripts/core/input.lua`
Category: Core Systems
Version: 1.0
Status: Implemented — Basic Binding Management

1. Overview

The Input module provides a lightweight system for associating named actions with numeric key or input identifiers.

It allows other systems to bind an action to an identifier, retrieve or check an existing binding, remove individual bindings, and clear all bindings belonging to an input instance.

The module currently manages binding data only. It does not read physical input, detect button presses, or process input events.

Key Features

- Create independent input-binding instances.
- Bind named actions to numeric identifiers.
- Remove individual action bindings.
- Retrieve an action's assigned identifier.
- Check whether an action has a binding.
- Clear all bindings in an instance.

Design Principles

The module provides a small data-management layer for action-to-key associations.

For example, an application might associate the action `"jump"` with a particular keyboard key identifier.

The Input module stores that association, while actual input detection remains the responsibility of Defold's input system or another system that consumes the binding data.

This separation avoids duplicating Defold's native input-event processing.

2. API Reference

The module exposes five public functions.

Function	Description
`new()`	Creates a new input-binding instance.
`bind(input, action_name, key)`	Associates an action with a numeric identifier.
`unbind(input, action_name)`	Removes an action's binding.
`get_binding(input, action_name)`	Retrieves an action's assigned identifier.
`has(input, action_name)`	Checks whether an action has a binding.
`clear(input)`	Removes all bindings from an instance.

2.1. `Input.new()`

Description

Creates a new input-binding instance.

Parameters

None.

Returns

A new table containing an empty `actions` table.

```
{
    actions = {}
}
```

Example

```
local Input = require "scripts.core.input"

local input = Input.new()
```

Important Notes

Each call creates a separate instance. Bindings stored in one instance do not automatically appear in another.

Creating an input-binding instance does not register physical keys or mouse buttons with Defold.

2.2. `Input.bind(input, action_name, key)`

Description

Associates a named action with a numeric key or input identifier.

If the action already has a binding, the existing value is replaced.

Parameters

Name	Type	Description
`input`	Input instance	The instance in which to store the binding.
`action_name`	`string`	The name of the action.
`key`	`number`	The numeric identifier associated with the action.

Returns

No explicit return value.

Example

```
local Input = require "scripts.core.input"

local input = Input.new()

Input.bind(input, "jump", hash("space"))
Input.bind(input, "attack",
hash("mouse_button_1"))
```

In this example, the identifiers are obtained using Defold's `hash()` function.

Behavior

The function stores the identifier under the supplied action name:

```
input.actions[action_name] = key
```

If the action already exists, its previous identifier is overwritten.

Validation

The function asserts that:

- `input` is not `nil`.
- `action_name` is a string.
- `key` is a number.

Important Notes

The module does not verify whether the numeric identifier corresponds to a valid physical input.

It also does not configure Defold's Input Binding resource or automatically connect the stored binding to `on_input()`.

2.3. `Input.unbind(input, action_name)`

Description

Removes the binding associated with a specific action.

Parameters

Name	Type	Description
`input`	Input instance	The instance from which to remove the binding.
`action_name`	`string`	The name of the action to unbind.

Returns

No explicit return value.

Example

```
Input.unbind(input, "jump")
```

Behavior

The function removes the action's entry by assigning `nil` to it.

If the action does not exist, the operation has no effect.

Validation

The function asserts that:

- `input` is not `nil`.
- `action_name` is a string.

2.4. `Input.get_binding(input, action_name)`

Description

Retrieves the numeric identifier associated with an action.

Parameters

Name	Type	Description
`input`	Input instance	The instance to query.
`action_name`	`string`	The name of the action.

Returns

- The numeric identifier if the action is bound.
- `nil` if the action has no binding.

Example

```
local jump_key = Input.get_binding(input,
"jump")

if jump_key ~= nil then
    print("Jump has a binding:", jump_key)
end
```

Validation

The function asserts that:

- `input` is not `nil`.
- `action_name` is a string.

Important Notes

The function only retrieves stored data. It does not determine whether the corresponding key is currently pressed or released.

2.5. `Input.has(input, action_name)`

Description

Checks whether an action has an assigned binding.

Parameters

Name	Type	Description
`input`	Input instance	The instance to query.
`action_name`	`string`	The name of the action.

Returns

`boolean`

- `true` if the action has a non-`nil` binding.
- `false` otherwise.

Example

```
if Input.has(input, "attack") then
    print("The attack action has a binding.")
end
```

Validation

The function asserts that:

- `input` is not `nil`.
- `action_name` is a string.

Important Notes

This function checks whether a binding exists, not whether the associated physical input is available or active.

2.6. `Input.clear(input)`

Description

Removes all action bindings from the specified input instance.

Parameters

Name	Type	Description
`input`	Input instance	The instance whose bindings should be cleared.

Returns

No explicit return value.

Example

```
Input.clear(input)
```

Behavior

The function replaces the instance's `actions` table with a new empty table.

Other input instances remain unaffected.

Validation

The function asserts that `input` is not `nil`.

3. Internal Implementation

3.1. Data Structure

Each input instance stores its bindings in an `actions` table.

For example:

```
{
    actions = {
        jump = hash("space"),
        attack = hash("mouse_button_1"),
        dash = hash("shift")
    }
}
```

This is an illustrative representation of the stored data.

The actual identifiers depend on the values supplied to `Input.bind()`.

3.2. Instance Independence

The module does not use a shared module-level binding table.

Each call to `Input.new()` creates a separate instance with its own `actions` table.

Consequently, modifying one instance does not directly modify another instance's bindings.

3.3. Binding Replacement

An action name can have only one stored identifier at a time.

Calling `Input.bind()` repeatedly with the same action name replaces the previous identifier.

For example:

```
Input.bind(input, "jump", hash("space"))
Input.bind(input, "jump", hash("w"))
```

After these calls, the stored binding for `"jump"` is the identifier returned by `hash("w")`.

The module does not currently support multiple identifiers for a single action.

4. Limitations

The current implementation has the following limitations:

Limitation	Description
No runtime input detection	The module does not detect physical input events.
No pressed-state tracking	It does not track whether a key is pressed, held, or released.
No automatic Defold integration	Stored bindings do not automatically configure Defold's Input Binding resource.
One identifier per action	Each action stores only one numeric identifier.
No binding validation	Numeric identifiers are accepted without checking whether they represent valid inputs.
No conflict detection	Multiple actions can be assigned the same identifier.

No automatic persistence	Bindings are not saved to a file by this module.
No automatic default bindings	The module does not define default actions or key assignments.

These limitations reflect the current scope of the module.

Runtime input processing should be considered separately when gameplay systems require it. Defold already provides input bindings and input-event callbacks, so any future extension should complement those features rather than unnecessarily duplicate them.

5. Quick Reference

Operation	Usage
Create an input instance	`Input.new()`
Bind an action	`Input.bind(input, "jump", key)`
Remove a binding	`Input.unbind(input, "jump")`
Retrieve a binding	`Input.get_binding(input, "jump")`
Check whether a binding exists	`Input.has(input, "jump")`
Clear all bindings	`Input.clear(input)`

6. Usage Guidelines

- Use descriptive action names such as `"jump"`, `"attack"`, and `"dash"`.
- Use numeric identifiers consistently.
- Retrieve bindings through `Input.get_binding()` rather than accessing the internal table directly when using the public API.
- Check `Input.has()` when the existence of a binding matters.
- Use `Input.unbind()` to remove one action without affecting others.
- Use `Input.clear()` only when every binding in the instance should be removed.
- Keep binding storage separate from runtime input-event processing.
- Do not assume that storing a binding automatically registers it with Defold.

7. Summary

The Input module currently provides a basic action-to-identifier binding table.

It supports:

- Independent input-binding instances.
- Action binding and replacement.
- Individual binding removal.
- Binding retrieval.
- Binding existence checks.
- Clearing all bindings in an instance.

It does not currently process input events or track key states.

Document status: `Draft 1.0` — based on the supplied implementation of `scripts/core/input.lua`.

Save System

File: `scripts/gameplay/save.lua`
Category: Gameplay Systems
Version: 1.0
Status: Implemented

1. Overview

The Save module provides a simple interface for managing key-value data and persisting that data to disk through Defold's built-in system API.

It allows callers to create save-data containers, set and retrieve values, remove entries, write data to a save file, and load data from a file.

The module stores application data in a Lua table and delegates file persistence to Defold.

Key Features

- Create independent save-data containers.
- Store values under string keys.
- Retrieve stored values.
- Remove individual entries.
- Save the data table to a file.
- Load data from a file.
- Check whether a loaded file contains any entries.

Design Principles

The Save module provides basic data storage and persistence without defining what the saved data represents.

For example, a game might store:

- Player progress.
- Unlocked abilities.
- Collected items.
- Current checkpoint information.
- Gameplay settings.

These are possible uses, not predefined fields managed by the module.

The module does not automatically connect save data to gameplay systems. The calling code is responsible for deciding which values to store and when to save or load them.

2. API Reference

The module exposes six public functions.

Function	Description
`new()`	Creates a new save-data container.
`set(save, key, value)`	Stores a value under a string key.
`get(save, key)`	Retrieves a stored value.
`remove(save, key)`	Removes an entry from the data table.
`save_to_file(save, filename)`	Writes the data table to a file.
`load_from_file(save, filename)`	Loads data from a file into the container.
`exists(filename)`	Checks whether the loaded data contains at least one entry.

2.1. `Save.new()`

Description

Creates a new save-data container.

Parameters

None.

Returns

A table containing a version field and an empty data table.

```
{
    version = 1,
    data = {}
}
```

Example

```
local Save = require "scripts.gameplay.save"

local save = Save.new()
```

Important Notes

The `version` field is initialized to `1`, but the current implementation does not use it for version validation, migration, or compatibility checks.

Additionally, only `save.data` is written to disk by `save_to_file()`. The `version` field is not included in the saved file by this module.

2.2. `Save.set(save, key, value)`

Description

Stores a value in the save-data container under the specified key.

If the key already exists, its previous value is replaced.

Parameters

Name	Type	Description
`save`	Save container	The container to modify.
`key`	`string`	The key associated with the value.
`value`	Any Lua value supported by the intended storage workflow	The value to store.

Returns

No explicit return value.

Example

```
Save.set(save, "player_level", 5)
Save.set(save, "has_dash", true)
Save.set(save, "current_room",
"forgotten_cavern")
```

Behavior

The function assigns the supplied value to `save.data[key]`.

Validation

The function asserts that:

- `save` is not `nil`.
- `key` is a string.

The function does not validate the value's type or whether it is serializable by Defold's save API.

Important Notes

Although the function accepts arbitrary Lua values without additional checks, the values must be compatible with the underlying persistence mechanism if they are to be saved successfully.

2.3. `Save.get(save, key)`

Description

Retrieves the value stored under the specified key.

Parameters

Name	Type	Description
`save`	Save container	The container to query.
`key`	`string`	The key to retrieve.

Returns

- The stored value if the key exists.
- `nil` if the key has no stored value.

Example

```
local player_level = Save.get(save,
"player_level")

if player_level ~= nil then
    print("Player level:", player_level)
end
```

Validation

The function asserts that:

- `save` is not `nil`.
- `key` is a string.

Important Notes

The function does not provide a default value when a key is missing.

The caller must handle `nil` when the requested key may not exist.

2.4. `Save.remove(save, key)`

Description

Removes a key-value entry from the save-data container.

Parameters

Name	Type	Description
`save`	Save container	The container to modify.
`key`	`string`	The key to remove.

Returns

No explicit return value.

Example

```
Save.remove(save, "temporary_flag")
```

Behavior

The function assigns `nil` to `save.data[key]`.

If the key does not exist, the operation has no effect.

Validation

The function asserts that:

- `save` is not `nil`.
- `key` is a string.

Important Notes

Removing an entry from memory does not immediately update a save file on disk.

To persist the removal, the caller must subsequently invoke `Save.save_to_file()`.

2.5. `Save.save_to_file(save, filename)`

Description

Writes the save container's `data` table to a file using Defold's `sys.save()` API.

Parameters

Name	Type	Description
`save`	Save container	The container whose data should be saved.
`filename`	`string`	The filename or supported save path used by Defold.

Returns

No explicit return value from the wrapper.

The function does not inspect or return the result of `sys.save()`.

Example

```
Save.set(save, "player_level", 5)
Save.set(save, "has_dash", true)

Save.save_to_file(save, "player_save")
```

Behavior

The function passes `save.data` directly to `sys.save()`:

```
sys.save(filename, save.data)
```

Validation

The function asserts that:

- `save` is not `nil`.
- `filename` is a string.

Important Notes

- Only `save.data` is passed to Defold.
- The container's `version` field is not saved.
- The function does not create a backup or maintain multiple save slots automatically.
- The function does not implement its own error handling or recovery mechanism.
- The caller is responsible for choosing an appropriate filename and deciding when saving should occur.

2.6. `Save.load_from_file(save, filename)`

Description

Loads data from a file using Defold's `sys.load()` API and replaces the container's current data table if the loaded table contains at least one entry.

Parameters

Name	Type	Description
`save`	Save container	The container into which the data should be loaded.
`filename`	`string`	The filename or supported save path used by Defold.

Returns

No explicit return value.

Example

```
local save = Save.new()

Save.load_from_file(save, "player_save")

local player_level = Save.get(save,
"player_level")
```

Behavior

1. Validates the save container and filename.
2. Calls `sys.load(filename)`.
3. Checks whether the returned table contains any entries.
4. Replaces `save.data` with the loaded table if it is non-empty.
5. Leaves the existing `save.data` unchanged if the loaded table is empty.

Validation

The function asserts that:

- `save` is not `nil`.
- `filename` is a string.

Important Notes

The current implementation assumes that `sys.load(filename)` returns a table on which `next()` can be called.

It does not explicitly handle load errors or validate the returned data structure.

If the loaded table is empty, the function preserves the container's existing data instead of clearing it.

For example:

```
local save = Save.new()

Save.set(save, "player_level", 5)

Save.load_from_file(save, "empty_save")
```

If the load operation returns an empty table, the in-memory `player_level` value remains `5`.

This behavior may be useful in some situations, but it also means that loading an empty save does not reset the current container.

2.7. `Save.exists(filename)`

Description

Loads data from the specified file and checks whether the returned table contains at least one entry.

Parameters

Name	Type	Description
`filename`	`string`	The filename or supported save path used by Defold.

Returns

`boolean`

- `true` if the loaded data contains at least one entry.
- `false` if the loaded table is empty.

Example

```
if Save.exists("player_save") then
    print("The save contains data.")
end
```

Behavior

The function calls `sys.load(filename)` and evaluates:

```
next(data) ~= nil
```

Validation

The function asserts that `filename` is a string.

Important Notes

Despite its name, this function does not directly check whether a physical file exists.

It checks whether loading the specified filename produces a table containing at least one entry.

Consequently, an empty save-data table produces `false`, even if a corresponding file exists.

The function also assumes that `sys.load()` returns a table. It does not explicitly handle errors or unexpected return values.

3. Internal Implementation

3.1. Data Structure

The Save module separates its container metadata from the actual stored data.

Conceptually, a save container may look like this:

```
{
    version = 1,

    data = {
        player_level = 5,
        has_dash = true,
        current_room = "forgotten_cavern"
    }
}
```

The example values are illustrative.

The current implementation persists only the `data` table, not the complete container.

3.2. In-Memory Storage

The functions `set()`, `get()`, and `remove()` operate on the container's in-memory `data` table.

Changes remain in memory until `save_to_file()` is called.

Removing a key or changing its value does not automatically write the changes to disk.

3.3. Persistence Through Defold

The module delegates persistence to two Defold functions:

- `sys.save()` writes data to the specified save location.
- `sys.load()` loads data from the specified save location.

The module does not implement its own serialization format or file-management mechanism.

3.4. Loading Behavior

The current loading logic replaces the container's data only when the loaded table is non-empty.

Conceptually:

```
Save.load_from_file(save, filename)
                  |
                  v
          Load data using sys.load()
                  |
                  v
          Does the table contain data?
             /             \
           Yes              No
            |                |
            v                v
 Replace save.data     Keep existing
 with loaded data      save.data
```

This behavior should be considered when reusing a save container to load different files.

4. Limitations

The current implementation has the following limitations:

Limitation	Description
Version metadata is not persisted	The `version` field is not included in the data written to disk.
No version migration	The module does not migrate older save formats.
No schema validation	Loaded data is not checked against an expected structure.
Empty loads preserve existing data	An empty loaded table does not replace the current data table.
No explicit load-error handling	The function assumes `sys.load()` returns a usable table.
No direct file-existence check	`exists()` checks for non-empty loaded data rather than physical file existence.
No automatic saving	Changes must be persisted by calling `save_to_file()`.
No backup or recovery system	The module does not maintain backup files or recover corrupted data.
No automatic gameplay integration	Other systems must explicitly provide and consume save data.

These limitations describe the current implementation, not necessarily requirements for future development.

Any extensions should be based on actual save-system requirements rather than added preemptively.

5. Quick Reference

Operation	Usage
Create a save container	`Save.new()`
Store a value	`Save.set(save, "key", value)`
Retrieve a value	`Save.get(save, "key")`
Remove a value	`Save.remove(save, "key")`

Save data to a file	`Save.save_to_file(save, "filename")`
Load data from a file	`Save.load_from_file(save, "filename")`
Check for non-empty loaded data	`Save.exists("filename")`

6. Usage Guidelines

- Use descriptive string keys for stored values.
- Keep the save container as the in-memory representation of the data.
- Call `save_to_file()` when changes need to persist.
- Handle missing values returned by `get()`.
- Remember that `remove()` changes memory only until the updated data is saved.
- Do not assume that `exists()` proves a physical save file exists.
- Do not assume that loading an empty table clears the current container.
- Keep the intended save-data structure consistent across the systems that read and write it.
- Consider versioning and migration only when the project actually requires compatibility between different save formats.

7. Summary

The Save module provides a basic key-value storage interface backed by Defold's persistence API.

It supports:

- Creating save containers.
- Setting, retrieving, and removing values.
- Saving data to a file.
- Loading data from a file.
- Checking whether loaded data is non-empty.

The module currently provides basic persistence rather than a complete save-management framework.

Document status: `Draft 1.0` — based on the supplied implementation of `scripts/gameplay/save.lua`.

Ability System

File: `scripts/gameplay/ability.lua`
Category: Gameplay Systems
Status: Implemented

1. Overview

The Ability System provides a basic structure for managing individual gameplay abilities.

Each ability stores its name, unlock status, enabled status, active status, cooldown duration, remaining cooldown, and an optional callback executed when the ability is used.

The system allows other gameplay systems to control whether an ability can be used, activate or stop it, configure its cooldown, and update its remaining cooldown over time.

The module requires the Core Timer System, but the current implementation does not use it.

2. Main Features

- Create independent ability instances.
- Lock and unlock abilities.
- Enable and disable abilities.
- Check whether an ability is available for use.
- Use abilities through an optional callback.
- Track whether an ability is active.
- Configure and track cooldowns.
- Update cooldowns using delta time.

3. API Reference

3.1 `Ability.new(name, on_use)`

Creates and returns a new ability instance.

Parameters

Parameter	Type	Required	Description
`name`	string	Yes	The ability's name.
`on_use`	function or `nil`	No	The callback executed when the ability is successfully used.

Initial values

Field	Initial value	Description
`name`	Provided name	Identifies the ability.
`unlocked`	`false`	Whether the ability has been unlocked.
`enabled`	`true`	Whether the ability is enabled.
`active`	`false`	Whether the ability is currently marked as active.
`cooldown`	`0`	The configured cooldown duration.
`cooldown_remaining`	`0`	The remaining cooldown duration.
`on_use`	Provided callback or `nil`	The function called when the ability is used.

Validation

- `name` must be a string.
- `on_use` must be a function or `nil`.

Example

```
local Ability = require
"scripts.gameplay.ability"

local dash = Ability.new("dash", function(data)
    print("Dash used")
end)
```

The new ability is locked by default, so it cannot be used until unlocked.

3.2 `Ability.unlock(ability)`

Unlocks the specified ability.

Parameter

- `ability`: The ability instance.

Behavior

Sets `ability.unlocked` to `true`.

This does not automatically activate the ability or reset its cooldown.

```
Ability.unlock(dash)
```

3.3 `Ability.lock(ability)`

Locks the specified ability.

Behavior

Sets `ability.unlocked` to `false`.

A locked ability cannot be used through `Ability.use()`.

Locking an ability does not automatically stop it if it is already active.

```
Ability.lock(dash)
```

3.4 `Ability.enable(ability)`

Enables the specified ability.

Behavior

Sets `ability.enabled` to `true`.

Enabling an ability does not unlock it.

```
Ability.enable(dash)
```

3.5 `Ability.disable(ability)`

Disables the specified ability.

Behavior

Sets `ability.enabled` to `false`.

A disabled ability cannot be used through `Ability.use()`.

Disabling an ability does not automatically stop it if it is already active.

```
Ability.disable(dash)
```

3.6 `Ability.can_use(ability)`

Checks whether an ability is unlocked and enabled.

Returns

- `true`: The ability is both unlocked and enabled.
- `false`: The ability is locked or disabled.

Important: This function does not check the remaining cooldown. An ability can return `true` from `can_use()` while still being unavailable because of its cooldown.

```
if Ability.can_use(dash) then
    print("The ability is unlocked and enabled")
end
```

3.7 `Ability.use(ability, data)`

Attempts to use an ability.

Parameters

Parameter	Type	Required	Description
`ability`	table	Yes	The ability instance.
`data`	Any	No	Data passed to the ability's callback.

Behavior

The function performs the following steps:

1. Checks whether the ability is unlocked and enabled.
2. Checks whether `cooldown_remaining` is greater than zero.
3. If either check fails, returns `false` without using the ability.
4. Marks the ability as active.
5. Sets `cooldown_remaining` to the configured `cooldown`.
6. Executes `on_use(data)` if a callback exists.
7. Returns `true`.

Returns

- `true`: The ability passed the checks and its use sequence completed.
- `false`: The ability was locked, disabled, or still on cooldown.

Example

```
local success = Ability.use(dash, {
    direction = 1
})

if success then
    print("Ability used successfully")
end
```

Important implementation details

- The cooldown starts immediately when the ability is used.
- The callback receives only the `data` argument. The ability instance is not passed automatically.
- If no callback exists, the ability can still be used successfully.
- The function does not automatically stop the ability after a duration.
- If the callback raises an error, the error propagates and the function does not reach its final `return true`. The ability has already been marked active and its cooldown has already been set.

3.8 `Ability.stop(ability)`

Marks the ability as inactive.

Behavior

Sets `ability.active` to `false`.

```
Ability.stop(dash)
```

This function does not reset or cancel the cooldown. The ability may remain unavailable until its cooldown reaches zero.

3.9 `Ability.set_cooldown(ability, duration)`

Sets the ability's cooldown duration.

Parameters

Parameter	Type	Required	Description
`ability`	table	Yes	The ability instance.
`duration`	number	Yes	The cooldown duration.

Validation

- `duration` must be a number.
- `duration` cannot be negative.

Behavior

Sets `ability.cooldown` to `duration`.

The function does not modify `cooldown_remaining`. Changing the configured cooldown therefore does not automatically adjust a cooldown already in progress.

```
Ability.set_cooldown(dash, 1.5)
```

The duration is measured in the same time units used by `dt`, normally seconds in a Defold update loop.

3.10 `Ability.update(ability, dt)`

Updates the ability's remaining cooldown.

Parameters

Parameter	Type	Required	Description
`ability`	table	Yes	The ability instance.
`dt`	number	Yes	The elapsed time since the previous update.

Behavior

If `cooldown_remaining` is greater than zero, the function subtracts `dt` and clamps the result to a minimum of zero.

```
function update(self, dt)
    Ability.update(self.dash, dt)
end
```

Important implementation details

- The function must be called repeatedly by the system managing the ability.
- It does not use Defold's Timer API.
- It does not validate that `dt` is nonnegative.
- It does not automatically change `active` when the cooldown expires.
- Updating the cooldown and stopping the ability are separate operations.

3.11 `Ability.get_cooldown(ability)`

Returns the configured cooldown duration.

```
local cooldown = Ability.get_cooldown(dash)
```

The returned value is `ability.cooldown`.

3.12 `Ability.get_cooldown_remaining(ability)`

Returns the remaining cooldown duration.

```
local remaining =
Ability.get_cooldown_remaining(dash)
```

The returned value is `ability.cooldown_remaining`.

A value of `0` means that the ability is not currently blocked by its cooldown.

4. Typical Usage Example

The following example demonstrates how to create, configure, unlock, use, update, and stop an ability.

```
local Ability = require
"scripts.gameplay.ability"

function init(self)
    self.dash = Ability.new("dash",
function(data)
        print("Dash direction:", data.direction)
    end)

    Ability.set_cooldown(self.dash, 1.5)
    Ability.unlock(self.dash)
end

function update(self, dt)
    Ability.update(self.dash, dt)
end

function on_dash_input(self, direction)
    local success = Ability.use(self.dash, {
        direction = direction
    })

    if success then
        print("Dash started")
    else
        print("Dash is unavailable")
    end
end

function stop_dash(self)
    Ability.stop(self.dash)
end
```

This example assumes that `on_dash_input()` and `stop_dash()` are called by other code when appropriate. The Ability System itself does not detect input, implement dash movement, or determine when the dash animation ends.

5. Implementation Notes

5.1 Ability Instances

Each call to `Ability.new()` returns a separate table. Each instance maintains its own state and cooldown values.

The module does not maintain a central registry of abilities.

5.2 Availability Rules

An ability can be used only when all three conditions are satisfied:

1. It is unlocked.
2. It is enabled.
3. Its remaining cooldown is zero.

The first two conditions are checked by `Ability.can_use()`. The cooldown condition is checked separately by `Ability.use()`.

5.3 Active State

The `active` field is a manually managed Boolean flag.

`Ability.use()` sets it to `true`, and `Ability.stop()` sets it to `false`. No other function automatically resets it.

Consequently, an ability can remain active after its cooldown expires, or remain active after being locked or disabled.

5.4 Cooldown Management

The cooldown is managed using the caller-provided delta time.

For example, if an ability has a cooldown of `1.5` seconds, its remaining cooldown decreases as `Ability.update()` is called with successive `dt` values.

The system does not create timers, schedule callbacks, or automatically perform actions when a cooldown expires.

5.5 Timer Dependency

The module contains the following import:

```
local Timer = require "scripts.core.timer"
```

However, `Timer` is not used anywhere in the current implementation. Cooldown management is performed directly through `Ability.update()`.

The import can be removed unless a future implementation requires the Timer System.

5.6 Error Handling

The public functions validate that the supplied ability is not `nil`. The system does not comprehensively validate the structure of an ability table created or modified externally.

Callback errors are not caught or handled by the module.

6. Limitations

The current implementation does not provide:

- Automatic ability activation or deactivation based on duration.
- Automatic stopping when the cooldown expires.
- Input handling or input-to-ability mapping.
- Animation, movement, combat, or visual-effect integration.
- Resource-cost management, such as stamina or mana.
- Charges or multiple uses before cooldown.
- Automatic cooldown updates; the caller must invoke `Ability.update()`.
- A central ability manager or ability registry.
- State-transition events or cooldown-completion callbacks.
- Validation of externally modified ability fields.
- Automatic use of the imported Timer System.

These features would require additional implementation or integration with other systems.

7. Quick Reference

Function	Purpose
`Ability.new(name, on_use)`	Creates an ability.
`Ability.unlock(ability)`	Unlocks the ability.
`Ability.lock(ability)`	Locks the ability.
`Ability.enable(ability)`	Enables the ability.
`Ability.disable(ability)`	Disables the ability.

`Ability.can_use(ability)`	Checks unlock and enabled status.
`Ability.use(ability, data)`	Attempts to use the ability.
`Ability.stop(ability)`	Marks the ability as inactive.
`Ability.set_cooldown(ability, duration)`	Sets the cooldown duration.
`Ability.update(ability, dt)`	Decreases the remaining cooldown.
`Ability.get_cooldown(ability)`	Gets the configured cooldown.
`Ability.get_cooldown_remaining(ability)`	Gets the remaining cooldown.

8. Usage Guidelines

- Create a separate ability instance for each ability that requires independent state.
- Unlock an ability before attempting to use it.
- Use `enabled` to control whether an unlocked ability is temporarily available.
- Call `Ability.update()` regularly while the ability's cooldown needs to progress.
- Call `Ability.stop()` when the gameplay logic determines that the ability should no longer be active.
- Do not treat `Ability.can_use()` as a complete cooldown check.
- Keep actual gameplay behavior, such as dash movement or attack hit detection, in the appropriate gameplay code.
- Remove the unused Timer import unless it is needed by a future implementation.

9. Summary

The Ability System is a basic per-ability state and cooldown manager. It provides the essential operations for locking, unlocking, enabling, disabling, using, stopping, and updating abilities.

It deliberately does not implement the gameplay effects of an ability. Other systems remain responsible for input, movement, animation, combat, and the conditions that determine when an ability should stop.

Ability Manager

File: `scripts/gameplay/ability_manager.lua`
Category: Gameplay Systems
Status: Implemented

1. Overview

The Ability Manager manages a collection of ability instances and provides a slot-based interface for accessing and using them.

It works alongside the Ability System (`scripts/gameplay/ability.lua`) rather than implementing ability behavior itself.

The manager stores abilities by name, allows abilities to be retrieved or removed, assigns abilities to numbered slots, and forwards slot-use requests to the Ability System. It also updates every ability registered with the manager.

2. Main Features

- Create independent ability managers.
- Register abilities by name.
- Retrieve abilities and check whether they exist.
- Remove abilities and clear their slot assignments.
- Assign abilities to numbered slots.
- Retrieve or clear slot assignments.
- Use an ability through its assigned slot.
- Update the cooldown of every registered ability.

3. Data Structure

Each manager instance contains two tables:

```
{
    abilities = {},
    slots = {}
}
```

`abilities`

Stores ability instances using their names as keys.

For example:

```
manager.abilities["dash"] = dash_ability
manager.abilities["double_jump"] =
double_jump_ability
```

`slots`

Stores the ability name assigned to each numeric slot.

For example:

```
manager.slots[1] = "dash"
manager.slots[2] = "double_jump"
```

Slots store ability names rather than direct references to ability instances. When a slot is used, the manager looks up the corresponding ability in `manager.abilities`.

4. API Reference

4.1 `AbilityManager.new()`

Creates and returns a new ability manager.

Returns

A table containing empty `abilities` and `slots` tables.

Example

```
local AbilityManager = require
"scripts.gameplay.ability_manager"

local manager = AbilityManager.new()
```

Each call creates a separate manager instance with independent collections.

4.2 `AbilityManager.add(manager, ability)`

Registers an ability with the manager.

Parameters

Parameter	Type	Description
`manager`	table	The manager instance.
`ability`	table	The ability instance to register.

Validation

- `manager` must not be `nil`.
- `ability` must not be `nil`.
- `ability.name` must be a string.

Behavior

Stores the ability using its name as the key:

```
manager.abilities[ability.name] = ability
```

Example

```
local Ability = require
"scripts.gameplay.ability"

local dash = Ability.new("dash")
AbilityManager.add(manager, dash)
```

Important: If an ability with the same name already exists, the new assignment replaces the previous entry. The manager does not prevent duplicate names or automatically clean up slot assignments that refer to a replaced ability.

4.3 `AbilityManager.get(manager, name)`

Returns the ability registered under the specified name.

Parameters

Parameter	Type	Description
`manager`	table	The manager instance.
`name`	string	The ability name.

Returns

- The registered ability instance, if found.
- `nil`, if no ability is registered under that name.

Example

```
local dash = AbilityManager.get(manager, "dash")

if dash then
    print("Dash exists")
end
```

4.4 `AbilityManager.has(manager, name)`

Checks whether an ability is registered under the specified name.

Returns

- `true`: An ability exists under that name.
- `false`: No ability exists under that name.

Example

```
if AbilityManager.has(manager, "dash") then
    print("Dash is registered")
end
```

This checks registration only. It does not check whether the ability is unlocked, enabled, active, or available after cooldown.

4.5 `AbilityManager.remove(manager, name)`

Removes an ability from the manager and clears every slot assigned to its name.

Behavior

1. Removes the ability from `manager.abilities`.
2. Iterates through `manager.slots`.
3. Removes every slot whose assigned ability name matches `name`.

Example

```
AbilityManager.remove(manager, "dash")
```

Important implementation details

- The function does not require the ability to exist before removal.
- If the name is not registered, any slot assignments using that name are still cleared.
- The function does not modify the removed ability instance itself.
- Other references to the removed ability remain valid.
- The function does not automatically stop the ability or cancel its cooldown.

4.6 `AbilityManager.update(manager, dt)`

Updates all abilities currently registered with the manager.

Parameters

Parameter	Type	Description
`manager`	table	The manager instance.
`dt`	number	The elapsed time since the previous update.

Behavior

Iterates through `manager.abilities` and calls:

```
Ability.update(ability, dt)
```

for each registered ability.

This forwards cooldown updates to the Ability System.

Example

```
function update(self, dt)
    AbilityManager.update(self.ability_manager,
dt)
end
```

Important: The manager must be updated regularly by the surrounding game code. It does not register its own update callback with Defold.

The function validates that `dt` is a number, but it does not reject negative values.

4.7 `AbilityManager.assign_slot(manager, slot, ability_name)`

Assigns a registered ability to a numbered slot.

Parameters

Parameter	Type	Description
`manager`	table	The manager instance.
`slot`	number	The numeric slot identifier.
`ability_name`	string	The name of the ability to assign.

Validation

- `manager` must not be `nil`.
- `slot` must be a number.
- `ability_name` must be a string.
- The named ability must already exist in `manager.abilities`.

Behavior

Stores the ability name in the selected slot.

```
manager.slots[slot] = ability_name
```

Example

```
AbilityManager.assign_slot(manager, 1, "dash")
AbilityManager.assign_slot(manager, 2,
"double_jump")
```

If the slot already contains another ability, the new assignment replaces it.

The function does not check whether the ability is unlocked or enabled. A locked or disabled ability can still be assigned to a slot.

4.8 `AbilityManager.get_slot(manager, slot)`

Returns the ability name assigned to the specified slot.

Parameters

Parameter	Type	Description
`manager`	table	The manager instance.
`slot`	number	The numeric slot identifier.

Returns

- The assigned ability name, if the slot is occupied.
- `nil`, if the slot is empty.

Example

```
local ability_name =
AbilityManager.get_slot(manager, 1)

if ability_name then
    print("Slot 1 contains:", ability_name)
end
```

This function returns the name, not the ability instance.

4.9 `AbilityManager.clear_slot(manager, slot)`

Clears the specified slot.

Behavior

Sets the selected entry in `manager.slots` to `nil`.

```
AbilityManager.clear_slot(manager, 1)
```

The ability remains registered in `manager.abilities`. Clearing a slot only removes the slot assignment.

The function does not require the slot to be occupied.

4.10 `AbilityManager.use_slot(manager, slot, data)`

Attempts to use the ability assigned to the specified slot.

Parameters

Parameter	Type	Description
`manager`	table	The manager instance.
`slot`	number	The numeric slot identifier.
`data`	any	Optional data passed to the ability's callback.

Behavior

1. Retrieves the ability name assigned to the slot.
2. Returns `false` if the slot is empty.
3. Looks up the ability in `manager.abilities`.
4. Returns `false` if the ability cannot be found.
5. Calls `Ability.use(ability, data)` and returns its result.

Returns

- `true`: The underlying Ability System reports successful use.
- `false`: The slot is empty, the ability is missing, or the ability cannot be used because it is locked, disabled, or on cooldown.

Example

```
local success = AbilityManager.use_slot(manager,
1, {
    direction = 1
})

if success then
    print("Slot ability used")
end
```

Important: The manager does not implement ability effects. It delegates the actual use operation to `Ability.use()`.

Errors raised by the underlying ability callback are not caught by the manager.

5. Typical Usage Example

The following example demonstrates creating a manager, registering abilities, assigning slots, using an ability, and updating cooldowns.

```
local Ability = require
"scripts.gameplay.ability"
local AbilityManager = require
"scripts.gameplay.ability_manager"

function init(self)
    self.ability_manager = AbilityManager.new()

    local dash = Ability.new("dash",
function(data)
        print("Dash direction:", data.direction)
    end)

    local double_jump =
Ability.new("double_jump", function()
        print("Double jump used")
    end)

    Ability.set_cooldown(dash, 1.5)
    Ability.set_cooldown(double_jump, 0.5)

    Ability.unlock(dash)
```

```
    Ability.unlock(double_jump)

    AbilityManager.add(self.ability_manager,
dash)
    AbilityManager.add(self.ability_manager,
double_jump)


AbilityManager.assign_slot(self.ability_manager,
1, "dash")

AbilityManager.assign_slot(self.ability_manager,
2, "double_jump")
end

function update(self, dt)
    AbilityManager.update(self.ability_manager,
dt)
end

function on_dash_input(self)

AbilityManager.use_slot(self.ability_manager, 1,
{
        direction = 1
    })
end

function on_jump_input(self)

AbilityManager.use_slot(self.ability_manager, 2)
end
```

This example assumes that the input functions are invoked by other code. The Ability Manager does not detect input events or automatically connect slots to Defold input actions.

6. Implementation Notes

6.1 Relationship with the Ability System

The manager depends on:

```
local Ability = require
"scripts.gameplay.ability"
```

It delegates cooldown updates and ability use to that module.

The manager is responsible for organization and access; the Ability System is responsible for each ability's own state and use checks.

6.2 Slot Assignments Are Name-Based

Slots store ability names rather than ability tables.

This means the manager resolves the current ability associated with a name whenever `use_slot()` is called.

If an ability is removed, its assigned slots are cleared. If an ability is replaced by another instance with the same name through `add()`, existing slots continue to refer to that name and therefore resolve to the replacement instance.

6.3 Multiple Slots Can Reference the Same Ability

The current implementation does not restrict an ability to one slot.

For example:

```
AbilityManager.assign_slot(manager, 1, "dash")
AbilityManager.assign_slot(manager, 3, "dash")
```

Both slots can refer to the same ability instance.

Because the cooldown belongs to the ability instance rather than the slot, using the ability through one slot also affects its availability through the other slot.

6.4 Slot Identifiers

The manager validates that slot identifiers are numbers, but it does not require them to be positive integers.

The caller is responsible for using consistent slot identifiers.

6.5 Shared Versus Independent State

Each call to `AbilityManager.new()` creates separate `abilities` and `slots` tables.

Abilities can nevertheless be shared between managers if the same ability instance is added to more than one manager. In that case, both managers reference the same object, including its active state and cooldown.

7. Limitations

The current implementation does not provide:

- Automatic input handling or input-to-slot mapping.
- A maximum number of slots.
- Validation that slot identifiers are positive integers.
- Automatic unlocking or enabling of assigned abilities.
- Automatic removal of an ability's references from external systems.
- Automatic stopping of abilities when removed.
- Ability switching animations or interface integration.
- Cooldown tracking independent of the underlying Ability System.
- A restriction preventing the same ability from occupying multiple slots.
- Protection against replacing an existing ability with another instance of the same name.
- Error handling for failures raised by ability callbacks.

These features would require additional implementation or integration with other systems.

8. Quick Reference

Function	Purpose
`AbilityManager.new()`	Creates a manager.
`AbilityManager.add(manager, ability)`	Registers an ability by name.
`AbilityManager.get(manager, name)`	Retrieves an ability.
`AbilityManager.has(manager, name)`	Checks whether an ability is registered.
`AbilityManager.remove(manager, name)`	Removes an ability and clears matching slots.
`AbilityManager.update(manager, dt)`	Updates all registered abilities.
`AbilityManager.assign_slot(manager, slot, ability_name)`	Assigns an ability to a slot.
`AbilityManager.get_slot(manager, slot)`	Retrieves a slot's ability name.
`AbilityManager.clear_slot(manager, slot)`	Clears a slot assignment.
`AbilityManager.use_slot(manager, slot, data)`	Attempts to use the assigned ability.

9. Usage Guidelines

- Create one manager for each collection of abilities that needs independent organization.
- Register abilities before assigning them to slots.
- Unlock and enable abilities separately when they should become usable.

- Call `AbilityManager.update()` regularly to advance cooldowns.
- Use `use_slot()` when gameplay input should activate an ability through a slot.
- Keep slot identifiers consistent throughout the codebase.
- Remember that multiple slots can reference the same ability and share its cooldown.
- Keep movement, combat, animation, and other ability effects in their appropriate gameplay implementations.

10. Summary

The Ability Manager organizes ability instances by name and provides a slot-based interface for using them. It also forwards update calls to every registered ability so their cooldowns can progress.

Together, the Ability System and Ability Manager provide a basic foundation for managing gameplay abilities. The manager handles registration and slot assignments, while each ability controls its own availability, active flag, cooldown, and use callback.

Checkpoint System

File: `scripts/gameplay/checkpoint.lua`
Category: Gameplay Systems
Status: Implemented

1. Overview

The Checkpoint System stores a checkpoint consisting of a room name and a position.

It provides functions to create a checkpoint, set its location, retrieve its room and position, and clear its stored data.

The system only manages checkpoint data. It does not handle room transitions, player respawning, persistent storage, or checkpoint activation logic.

2. Main Features

- Create independent checkpoint instances.
- Store a room name.
- Store a position value.
- Retrieve the stored room name.
- Retrieve the stored position.
- Clear the checkpoint data.

3. Data Structure

Each checkpoint contains two fields:

```
{
    room = nil,
    position = nil
}
```

Field	Type	Initial value	Description
`room`	string or `nil`	`nil`	The name of the room associated with the checkpoint.
`position`	Any or `nil`	`nil`	The stored position associated with the checkpoint.

The `position` field has no enforced type. The system stores whatever value is supplied to `Checkpoint.set()`.

For example, the caller may supply a position represented by a Defold vector or another suitable data structure.

4. API Reference

4.1 `Checkpoint.new()`

Creates and returns a new checkpoint instance.

Returns

A table containing `room = nil` and `position = nil`.

Example

```
local Checkpoint = require
"scripts.gameplay.checkpoint"

local checkpoint = Checkpoint.new()
```

Each call creates an independent checkpoint table.

4.2 `Checkpoint.set(checkpoint, room_name, position)`

Sets the checkpoint's room and position.

Parameters

Parameter	Type	Required	Description
`checkpoint`	table	Yes	The checkpoint instance.
`room_name`	string	Yes	The name of the room.
`position`	Any	Yes	The position value to store.

Validation

- `checkpoint` must not be `nil`.
- `room_name` must be a string.

The position is not type-checked.

Behavior

Updates both fields:

```
checkpoint.room = room_name
checkpoint.position = position
```

Example

```
local position = vmath.vector3(120, 80, 0)

Checkpoint.set(checkpoint, "forgotten_ruins",
position)
```

Calling this function again replaces the previously stored room and position.

The function does not verify that the room exists or that the position is valid within that room.

4.3 `Checkpoint.get_room(checkpoint)`

Returns the checkpoint's stored room name.

Parameter

- `checkpoint`: The checkpoint instance.

Returns

The value of `checkpoint.room`, which may be a string or `nil`.

Example

```
local room_name =
Checkpoint.get_room(checkpoint)

if room_name then
    print("Checkpoint room:", room_name)
end
```

4.4 `Checkpoint.get_position(checkpoint)`

Returns the checkpoint's stored position.

Parameter

- `checkpoint`: The checkpoint instance.

Returns

The value of `checkpoint.position`, which may be any value, including `nil`.

Example

```
local position =
Checkpoint.get_position(checkpoint)

if position then
    print("Checkpoint position:", position)
end
```

The system returns the stored value directly. It does not create a copy or convert the position into another format.

4.5 `Checkpoint.clear(checkpoint)`

Clears the checkpoint's stored data.

Behavior

Sets both fields to `nil`:

```
checkpoint.room = nil
checkpoint.position = nil
```

Example

```
Checkpoint.clear(checkpoint)
```

After this call, the checkpoint no longer contains a room name or position.

The function does not delete the checkpoint instance itself.

5. Typical Usage Example

The following example demonstrates creating a checkpoint, assigning its location, and retrieving the stored data.

```
local Checkpoint = require
"scripts.gameplay.checkpoint"

function init(self)
    self.checkpoint = Checkpoint.new()

    local player_position = vmath.vector3(120,
80, 0)

    Checkpoint.set(
        self.checkpoint,
        "forgotten_ruins",
        player_position
    )
end

function respawn_player(self)
    local room_name =
Checkpoint.get_room(self.checkpoint)
    local position =
Checkpoint.get_position(self.checkpoint)

    if room_name and position then
        print("Respawn room:", room_name)
        print("Respawn position:", position)
    end
end
```

This example only retrieves the checkpoint data. The surrounding game code must implement room loading and player repositioning.

6. Implementation Notes

6.1 Data Storage

The checkpoint is stored in an ordinary Lua table.

The module does not use Defold's `sys.save()`, `sys.load()`, or any other persistent storage mechanism.

Unless another system saves the data, the checkpoint exists only in memory.

6.2 Position Representation

The system does not impose a particular representation for `position`.

The caller is responsible for supplying a value compatible with the code that later reads it.

The system also does not copy the position value. If a mutable table or object is supplied, the stored field references that same value.

6.3 Room Validation

The room name is validated only to ensure that it is a string.

The system does not check whether the named room exists, is loaded, or can be entered.

6.4 Validation Scope

Every public function checks that the checkpoint argument is not `nil`.

These checks do not guarantee that the supplied value is a valid checkpoint table. Passing an incompatible value may still cause a Lua error.

7. Limitations

The current implementation does not provide:

- Automatic checkpoint activation.
- Automatic player respawning.
- Room loading or room transitions.
- Player position restoration.
- Persistent saving or loading.
- Multiple named checkpoints managed by a registry.
- Validation of room existence.
- Validation or copying of position values.
- Checkpoint activation conditions, such as touching an object.
- Checkpoint-specific events or callbacks.

These responsibilities must be implemented by other systems if required.

8. Quick Reference

Function	Purpose
`Checkpoint.new()`	Creates a checkpoint instance.
`Checkpoint.set(checkpoint, room_name, position)`	Stores the room and position.
`Checkpoint.get_room(checkpoint)`	Retrieves the room name.
`Checkpoint.get_position(checkpoint)`	Retrieves the position.
`Checkpoint.clear(checkpoint)`	Clears the stored data.

9. Usage Guidelines

- Create a checkpoint instance before using the other functions.
- Use a consistent room-name convention throughout the project.
- Store a position representation that the respawn logic understands.
- Keep room transitions and player repositioning outside this module.
- Use a separate persistence system if checkpoints must survive application restarts.
- Ensure that other systems handle the case where the checkpoint has not been set or has been cleared.

10. Summary

The Checkpoint System is a lightweight data container for storing a room name and a position.

It provides the basic operations needed to set, retrieve, and clear checkpoint data without taking responsibility for respawning, room management, or persistence.

Health System

File: `scripts/gameplay/health.lua`
Category: Gameplay Systems
Status: Implemented

1. Overview

The Health System manages an entity's basic health state.

It supports creating a health instance, applying damage, restoring health, checking current and maximum health, checking whether the entity is dead, and reviving a dead entity.

The system only manages health values and the dead flag. It does not implement combat, damage sources, healing items, death animations, respawning, or player movement.

2. Main Features

- Create independent health instances.
- Configure maximum health.
- Apply damage without allowing health to fall below zero.
- Heal without exceeding maximum health.
- Prevent damage and healing while dead.
- Retrieve current and maximum health.
- Check whether the entity is dead.
- Revive a dead entity with a specified health amount.

3. Data Structure

Each health instance contains three fields:

```
{
    current = max_health,
    max = max_health,
    dead = false
}
```

Field	Type	Description
`current`	number	The entity's current health.
`max`	number	The entity's maximum health.
`dead`	boolean	Whether the entity is marked as dead.

When an instance is created, its current health equals its maximum health, and `dead` is set to `false`.

4. API Reference

4.1 `Health.new(max_health)`

Creates and returns a new health instance.

Parameter

Parameter	Type	Description
`max_health`	number	The entity's maximum health. Must be greater than zero.

Validation

- `max_health` must be a number.
- `max_health` must be greater than zero.

Returns

A new health table initialized to full health.

Example

```
local Health = require "scripts.gameplay.health"
```

```
local player_health = Health.new(7)
```

The resulting instance has:

- `current = 7`
- `max = 7`
- `dead = false`

The system does not require health values to be integers, so fractional values are accepted.

4.2 `Health.damage(health, amount)`

Applies damage to an entity.

Parameters

Parameter	Type	Description
`health`	table	The health instance.
`amount`	number	The amount of damage to apply. Must be greater than zero.

Validation

- `health` must not be `nil`.
- `amount` must be a number.
- `amount` must be greater than zero.

Behavior

1. Checks whether the entity is already dead.
2. If dead, returns `false` without applying damage.
3. Otherwise, subtracts `amount` from current health.
4. Clamps current health to a minimum of zero.
5. If current health reaches zero, sets `dead` to `true`.
6. Returns `true`.

Example

```
Health.damage(player_health, 2)
```

If current health is `7`, the result is:

- `current = 5`
- `dead = false`

If current health is `2` and the entity receives `3` damage, the result is:

- `current = 0`
- `dead = true`

Returns

- `true`: Damage was processed.
- `false`: The entity was already dead.

The return value does not indicate whether the damage caused death.

4.3 `Health.heal(health, amount)`

Restores an entity's health.

Parameters

Parameter	Type	Description
`health`	table	The health instance.

`amount`	number	The amount of health to restore. Must be greater than zero.

Validation

- `health` must not be `nil`.
- `amount` must be a number.
- `amount` must be greater than zero.

Behavior

1. Checks whether the entity is dead.
2. If dead, returns `false` without restoring health.
3. Otherwise, adds `amount` to current health.
4. Clamps current health to a maximum of `health.max`.
5. Returns `true`.

Example

```
Health.heal(player_health, 2)
```

If current health is `3` and maximum health is `7`, the result is:

- `current = 5`
- `dead = false`

If current health is `6` and the entity receives `3` healing, current health becomes `7`, not `9`.

Returns

- `true`: Healing was processed.
- `false`: The entity was already dead.

Healing does not revive a dead entity. Use `Health.revive()` for that operation.

4.4 `Health.get_current(health)`

Returns the entity's current health.

Example

```
local current_health =
Health.get_current(player_health)
```

The function returns the value of `health.current`.

4.5 `Health.get_max(health)`

Returns the entity's maximum health.

Example

```
local max_health = Health.get_max(player_health)
```

The function returns the value of `health.max`.

4.6 `Health.is_dead(health)`

Checks whether the entity is marked as dead.

Returns

- `true`: The `dead` field is `true`.
- `false`: The `dead` field is `false`.

Example

```
if Health.is_dead(player_health) then
    print("The player is dead")
end
```

This function reads the stored flag. It does not independently check whether current health is zero.

4.7 `Health.revive(health, amount)`

Revives an entity and assigns it a specified amount of health, limited by its maximum health.

Parameters

Parameter	Type	Description
`health`	table	The health instance.
`amount`	number	The health value to assign. Must be greater than zero.

Validation

- `health` must not be `nil`.
- `amount` must be a number.
- `amount` must be greater than zero.

Behavior

1. Sets current health to the smaller of `amount` and `health.max`.
2. Sets `dead` to `false`.
3. Returns `true`.

Example

```
Health.revive(player_health, 3)
```

If maximum health is `7`, the resulting state is:

- `current = 3`
- `max = 7`
- `dead = false`

If the supplied amount exceeds maximum health, current health is capped at the maximum.

Important: The function does not require the entity to be dead before calling it. It can also reset the health of an entity that is already alive.

Revival is not restricted to the transition from dead to alive; the function always assigns the requested health value and clears the dead flag.

5. Typical Usage Example

The following example demonstrates creating a health instance, applying damage, healing, checking for death, and reviving.

```
local Health = require "scripts.gameplay.health"

function init(self)
    self.health = Health.new(7)
end

function take_damage(self, amount)
    local processed = Health.damage(self.health,
amount)

    if not processed then
        return
    end

    if Health.is_dead(self.health) then
        print("Entity died")
    end
```

```
end

function heal_entity(self, amount)
    Health.heal(self.health, amount)
end

function revive_entity(self, amount)
    Health.revive(self.health, amount)
end
```

The example assumes that other gameplay code calls these functions when damage, healing, or revival should occur.

The Health System does not detect collisions or decide when damage should be applied.

6. Implementation Notes

6.1 Independent Instances

Each call to `Health.new()` creates a separate health table.

Different entities can therefore maintain independent health values.

6.2 Damage and Healing Limits

Damage cannot reduce current health below zero through `Health.damage()`.

Healing cannot increase current health beyond maximum health through `Health.heal()`.

Revival also caps the assigned health at the maximum.

These limits apply to the provided functions. Directly modifying the fields outside the module can bypass them.

6.3 Death State

The system marks an entity as dead when damage reduces current health to zero.

Once dead:

- `Health.damage()` returns `false`.
- `Health.heal()` returns `false`.
- `Health.revive()` can restore health and clear the dead flag.

The system does not automatically destroy, disable, animate, or respawn the entity.

6.4 Revival Semantics

`Health.revive()` assigns a new current health value rather than adding health to the existing value.

For example, reviving an entity with `amount = 3` sets current health to `3`, even if it previously had a different value.

The function also sets `dead = false` regardless of the entity's previous state.

6.5 Validation Scope

The functions validate that the health argument is not `nil`, but they do not comprehensively validate the structure of a supplied health table.

The numeric validation also does not explicitly reject non-finite numbers such as infinity or NaN.

The system does not enforce integer-only health values.

7. Limitations

The current implementation does not provide:

- Damage types or resistance calculations.
- Armor or damage reduction.
- Invulnerability frames.
- Damage-over-time effects.
- Automatic death events or callbacks.
- Automatic healing over time.
- Health regeneration.
- Healing item integration.
- Maximum-health upgrades.
- Automatic respawning or checkpoint integration.
- Health persistence.
- Validation against externally modified health tables.
- Automatic synchronization with a user interface.

These features would require additional implementation or integration with other gameplay systems.

8. Quick Reference

Function	Purpose
`Health.new(max_health)`	Creates a health instance at full health.
`Health.damage(health, amount)`	Applies damage and updates the death state.
`Health.heal(health, amount)`	Restores health without exceeding the maximum.
`Health.get_current(health)`	Returns current health.
`Health.get_max(health)`	Returns maximum health.
`Health.is_dead(health)`	Returns the stored death flag.
`Health.revive(health, amount)`	Assigns health and clears the death flag.

9. Usage Guidelines

- Create one health instance for each entity whose health must be tracked independently.
- Route damage and healing through the system's functions to preserve their built-in limits.
- Check the return value of `Health.damage()` or `Health.heal()` when the caller needs to know whether the operation was processed.
- Use `Health.is_dead()` to check the stored death state.
- Use `Health.revive()` when the entity should be restored from a dead state.
- Keep collision detection, damage sources, death animations, and respawn behavior in the appropriate gameplay systems.
- Avoid modifying `current`, `max`, or `dead` directly unless the calling code intentionally needs to bypass the system's normal behavior.

10. Summary

The Health System provides a minimal health-state implementation with damage, healing, death detection, and revival.

It enforces basic health limits and prevents damage or healing through its public functions while an entity is dead. It leaves combat rules, death handling, and respawning to other gameplay systems.

Inventory System

File: `scripts/gameplay/inventory.lua`
Category: Gameplay Systems
Status: Implemented

1. Overview

The Inventory System manages a collection of items and their quantities.

Each inventory instance stores item names as keys and their corresponding quantities as numeric values. The system supports adding items, removing items, checking whether an item exists, retrieving item counts, clearing the inventory, and accessing the complete item table.

The system manages inventory data only. It does not implement item definitions, equipment, item usage, inventory capacity, trading, or user interface integration.

2. Main Features

- Create independent inventory instances.
- Add items and accumulate their quantities.
- Remove items when sufficient quantities are available.
- Check whether an item exists in the inventory.
- Retrieve the quantity of a specific item.
- Clear all inventory contents.
- Retrieve the complete item table.

3. Data Structure

Each inventory instance contains an `items` table.

```
{
    items = {}
}
```

Items are stored using their names as keys and their quantities as values.

For example:

```
inventory.items["health_potion"] = 3
inventory.items["key"] = 2
inventory.items["coin"] = 50
```

This represents an inventory containing three health potions, two keys, and fifty coins.

Field	Type	Description
`items`	table	Maps item names to their quantities.
Item name	string	Identifies an item.
Item quantity	number	Stores the amount of that item.

Items with a quantity of zero are removed from the table rather than retained with a zero value.

4. API Reference

4.1 `Inventory.new()`

Creates and returns a new inventory instance.

Returns

A table containing an empty `items` table.

Example

```
local Inventory = require
"scripts.gameplay.inventory"

local inventory = Inventory.new()
```

Each call creates a separate inventory instance with its own item table.

4.2 `Inventory.add(inventory, item_name, amount)`

Adds a specified quantity of an item to the inventory.

Parameters

Parameter	Type	Description
`inventory`	table	The inventory instance.
`item_name`	string	The name of the item to add.
`amount`	number	The quantity to add. Must be greater than zero.

Validation

- `inventory` must not be `nil`.
- `item_name` must be a string.
- `amount` must be a number.
- `amount` must be greater than zero.

Behavior

If the item already exists, its quantity increases by `amount`.

If the item does not exist, its quantity is initialized to `amount`.

The operation is equivalent to:

```
inventory.items[item_name] =
    (inventory.items[item_name] or 0) + amount
```

Example

```
Inventory.add(inventory, "health_potion", 3)
Inventory.add(inventory, "health_potion", 2)
```

After these calls, the inventory contains five health potions.

Returns

The function does not explicitly return a value.

Important: The system does not impose a maximum quantity or inventory capacity.

4.3 `Inventory.remove(inventory, item_name, amount)`

Removes a specified quantity of an item from the inventory.

Parameters

Parameter	Type	Description
`inventory`	table	The inventory instance.
`item_name`	string	The name of the item to remove.
`amount`	number	The quantity to remove. Must be greater than zero.

Validation

- `inventory` must not be `nil`.
- `item_name` must be a string.

- `amount` must be a number.
- `amount` must be greater than zero.

Behavior

1. Checks whether the item exists.
2. Returns `false` if the item is absent.
3. Checks whether the inventory contains enough of the item.
4. Returns `false` if the available quantity is insufficient.
5. Subtracts the requested quantity.
6. Removes the item's entry if its quantity reaches exactly zero.
7. Returns `true`.

Example

```
local success = Inventory.remove(inventory,
"health_potion", 2)

if success then
    print("Items removed successfully")
else
    print("Item missing or insufficient
quantity")
end
```

If the inventory contains five health potions, removing two leaves three.

If it contains only one and the caller attempts to remove two, the operation fails without changing the quantity.

Returns

- `true`: The requested quantity was removed.
- `false`: The item does not exist or its quantity is insufficient.

The function does not allow removal of more items than are available.

4.4 `Inventory.has(inventory, item_name)`

Checks whether the inventory contains an entry for the specified item.

Parameters

Parameter	Type	Description
`inventory`	table	The inventory instance.
`item_name`	string	The item name to check.

Returns

- `true`: The item has an entry in the inventory.
- `false`: No entry exists for the item.

Example

```
if Inventory.has(inventory, "key") then
    print("The inventory contains a key")
end
```

The function checks whether the item entry is non-`nil`. Under normal use of this module, this means the item has a positive quantity.

4.5 `Inventory.get_count(inventory, item_name)`

Returns the quantity of a specified item.

Parameters

Parameter	Type	Description
`inventory`	table	The inventory instance.
`item_name`	string	The item name.

Returns

- The stored quantity if the item exists.
- `0` if the item has no entry.

Example

```
local potion_count =
Inventory.get_count(inventory, "health_potion")

print("Health potions:", potion_count)
```

This function provides a numeric result even when the item is absent.

4.6 `Inventory.clear(inventory)`

Removes all items from the inventory.

Behavior

Replaces the existing item table with a new empty table.

```
inventory.items = {}
```

Example

```
Inventory.clear(inventory)
```

After this call, the inventory contains no registered items.

Important: The inventory instance itself remains valid. However, any external references to the previous `inventory.items` table still reference that old table and will not automatically point to the new one.

4.7 `Inventory.get_items(inventory)`

Returns the inventory's complete item table.

Returns

The actual `inventory.items` table, not a copy.

Example

```
local items = Inventory.get_items(inventory)

for item_name, amount in pairs(items) do
    print(item_name, amount)
end
```

Important: Because the function returns the original table, callers can modify the inventory directly through the returned reference.

For example:

```
local items = Inventory.get_items(inventory)
items["health_potion"] = 10
```

This changes the inventory's stored quantity without calling `Inventory.add()`.

Direct modifications can bypass the module's validation and normal item-removal behavior.

5. Typical Usage Example

The following example demonstrates creating an inventory, adding items, checking quantities, removing items, and clearing the inventory.

```
local Inventory = require
"scripts.gameplay.inventory"

function init(self)
    self.inventory = Inventory.new()

    Inventory.add(self.inventory,
"health_potion", 3)
    Inventory.add(self.inventory, "key", 1)
    Inventory.add(self.inventory, "coin", 25)
end

function use_health_potion(self)
    local potion_count = Inventory.get_count(
        self.inventory,
        "health_potion"
    )

    if potion_count > 0 then
        local success = Inventory.remove(
            self.inventory,
            "health_potion",
            1
        )

        if success then
            print("Health potion consumed")
        end
    else
        print("No health potions available")
    end
end

function check_key(self)
    if Inventory.has(self.inventory, "key") then
        print("The player has a key")
    end
end

function clear_inventory(self)
    Inventory.clear(self.inventory)
end
```

This example demonstrates inventory data management only. It does not implement potion effects, unlocking doors, or any other gameplay behavior.

6. Implementation Notes

6.1 Item Identification

Items are identified by string names.

The system does not use unique item IDs, item definition objects, or item metadata.

Names must be consistent across the code that adds, checks, and removes items.

For example, `"health_potion"` and `"Health_Potion"` are different keys.

6.2 Quantity Management

The `add()` function accumulates quantities rather than replacing them.

The `remove()` function subtracts quantities only when enough items are available.

When a quantity reaches exactly zero through `remove()`, the corresponding key is deleted.

6.3 No Item Objects

The inventory stores numeric quantities rather than individual item instances.

Consequently, it does not distinguish between two copies of the same item that have different properties, durability, or other individual state.

6.4 No Capacity Limit

The system does not track inventory weight, maximum item counts, or available slots.

Items can be added without checking available capacity.

6.5 Validation Scope

The functions validate their required arguments and ensure that amounts are positive numbers.

They do not explicitly reject non-finite numeric values such as infinity or NaN. The system also does not require item quantities to be integers.

The inventory table's structure is not comprehensively validated, so externally modified data can cause unexpected behavior.

7. Limitations

The current implementation does not provide:

- Item definitions or metadata.
- Unique item identifiers.
- Individual item instances.
- Equipment slots.
- Item usage or consumption effects.
- Inventory capacity or weight limits.
- Item stacking rules beyond accumulating quantities.
- Sorting or filtering.
- Trading, buying, or selling.
- Inventory persistence.
- User interface integration.
- Item pickup or drop logic.
- Protection against direct modification through `get_items()`.

These features would require additional implementation or integration with other systems.

8. Quick Reference

Function	Purpose
`Inventory.new()`	Creates an empty inventory.
`Inventory.add(inventory, item_name, amount)`	Adds an item quantity.
`Inventory.remove(inventory, item_name, amount)`	Removes a quantity if sufficient items exist.
`Inventory.has(inventory, item_name)`	Checks whether an item exists.
`Inventory.get_count(inventory, item_name)`	Returns an item's quantity, or zero if absent.
`Inventory.clear(inventory)`	Removes all registered items.
`Inventory.get_items(inventory)`	Returns the underlying item table.

9. Usage Guidelines

- Create a separate inventory instance for each entity that requires its own item collection.
- Use consistent item-name strings throughout the project.
- Check the return value of `Inventory.remove()` before proceeding with gameplay logic that depends on successful removal.
- Use `Inventory.get_count()` when the caller needs a numeric quantity.
- Avoid modifying the table returned by `Inventory.get_items()` directly unless bypassing the module's normal operations is intentional.
- Keep item effects, pickup logic, equipment behavior, and interface updates in their appropriate systems.
- Add persistence separately if inventory contents must survive application restarts.

10. Summary

The Inventory System is a lightweight quantity-based item manager.

It provides the basic operations required to add, remove, inspect, and clear items while preventing normal removal operations from reducing quantities below zero.

It does not implement item behavior, capacity management, persistence, or user interface logic. These responsibilities remain with other gameplay systems.

Respawn System

File: `scripts/gameplay/respawn.lua`
Category: Gameplay Systems
Status: Implemented

1. Overview

The Respawn System provides a small interface for retrieving checkpoint information and checking whether a checkpoint can be used for respawning.

It depends on the Checkpoint System (`scripts/gameplay/checkpoint.lua`) and delegates checkpoint data retrieval to that module.

The system does not perform the respawn operation itself. It only exposes the checkpoint room, the checkpoint position, and a basic check indicating whether a room has been assigned.

2. Main Features

- Retrieve the checkpoint room.
- Retrieve the checkpoint position.
- Check whether the checkpoint has a room assigned.
- Reuse the Checkpoint System's existing data-access functions.

3. Dependency

The module imports:

```
local Checkpoint = require
"scripts.gameplay.checkpoint"
```

The Respawn System relies on the Checkpoint System to retrieve stored checkpoint information.

It does not maintain its own checkpoint data.

4. API Reference

4.1 `Respawn.get_room(checkpoint)`

Returns the room name stored in the checkpoint.

Parameter

Parameter	Type	Description
`checkpoint`	table	The checkpoint instance.

Validation

The checkpoint must not be `nil`.

Returns

The result of `Checkpoint.get_room(checkpoint)`, which may be a room-name string or `nil`.

Example

```
local Respawn = require
"scripts.gameplay.respawn"

local room_name = Respawn.get_room(checkpoint)

if room_name then
    print("Respawn room:", room_name)
end
```

The function does not load the room or move the player.

4.2 `Respawn.get_position(checkpoint)`

Returns the position stored in the checkpoint.

Parameter

Parameter	Type	Description
`checkpoint`	table	The checkpoint instance.

Validation

The checkpoint must not be `nil`.

Returns

The result of `Checkpoint.get_position(checkpoint)`.

The returned value may be `nil` if no position has been stored.

Example

```
local position =
Respawn.get_position(checkpoint)

if position then
    print("Respawn position:", position)
end
```

The function returns the stored position value without modifying or copying it.

4.3 `Respawn.can_respawn(checkpoint)`

Checks whether the checkpoint has a room assigned.

Parameter

Parameter	Type	Description
`checkpoint`	table	The checkpoint instance.

Validation

The checkpoint must not be `nil`.

Returns

- `true`: The checkpoint's room is not `nil`.
- `false`: The checkpoint's room is `nil`.

Example

```
if Respawn.can_respawn(checkpoint) then
    print("A respawn room is available")
else
    print("No respawn room has been assigned")
end
```

Important: This function checks only whether a room has been assigned. It does not verify that the checkpoint position exists, that the room is valid, or that the room can be loaded.

5. Typical Usage Example

The following example demonstrates checking whether respawning is possible and retrieving the checkpoint information needed by other gameplay code.

```
local Respawn = require
"scripts.gameplay.respawn"
```

```
function respawn_player(self)
    if not Respawn.can_respawn(self.checkpoint)
then
        print("Cannot respawn: no checkpoint
room")
        return
    end

    local room_name =
Respawn.get_room(self.checkpoint)
    local position =
Respawn.get_position(self.checkpoint)

    print("Respawn room:", room_name)
    print("Respawn position:", position)

    -- Room loading and player repositioning
    -- must be implemented by other systems.
end
```

This example retrieves the data but deliberately leaves the actual respawn operation to the surrounding gameplay code.

6. Implementation Notes

6.1 No Independent State

The Respawn System does not create or store a respawn state.

It reads data from an existing checkpoint instance.

6.2 Delegated Data Access

Both `get_room()` and `get_position()` call the corresponding functions in the Checkpoint System.

This avoids duplicating checkpoint data-access logic.

6.3 Respawn Eligibility

The current eligibility check is:

```
return Checkpoint.get_room(checkpoint) ~= nil
```

Consequently, a checkpoint can pass `can_respawn()` even when its position is `nil`.

The function also does not check whether the entity is dead. It checks the checkpoint only, not the entity's current state.

6.4 Error Handling

The public functions check that the checkpoint argument is not `nil`.

They do not validate the complete checkpoint structure or handle errors raised by the Checkpoint System.

7. Limitations

The current implementation does not provide:

- Actual player respawning.
- Room loading or transitions.
- Player repositioning.
- Integration with the Health System.
- Integration with the death state.
- Validation that the checkpoint position exists.
- Validation that the room exists.
- Respawn delays or animations.
- Respawn events or callbacks.
- Selection between multiple checkpoints.
- Automatic checkpoint discovery or activation.

These responsibilities must be implemented by other systems if required.

8. Quick Reference

Function	Purpose
`Respawn.get_room(checkpoint)`	Retrieves the checkpoint room.
`Respawn.get_position(checkpoint)`	Retrieves the checkpoint position.
`Respawn.can_respawn(checkpoint)`	Checks whether a room has been assigned.

9. Usage Guidelines

- Use the Checkpoint System to create and update checkpoint data.
- Use `Respawn.can_respawn()` before attempting to retrieve respawn information.
- Do not assume that `can_respawn()` guarantees a valid position or a loadable room.
- Keep room transitions and player repositioning in the appropriate gameplay or scene-management code.
- Integrate this module with the Health System or death-handling logic if respawning should occur automatically after death.

10. Summary

The Respawn System is a lightweight interface over the Checkpoint System.

It exposes checkpoint room and position information and provides a basic check for whether a respawn room has been assigned. Actual respawning remains the responsibility of other gameplay systems.

Room Manager

File: `scripts/gameplay/room_manager.lua`
Category: Gameplay Systems
Status: Implemented

1. Overview

The Room Manager stores and manages the name of the currently active room.

It provides a simple interface for setting the current room, retrieving its name, checking whether a specific room is current, and clearing the stored room reference.

The system manages room identification only. It does not load rooms, perform transitions, or interact with game objects.

2. Features

- Stores the current room name.
- Sets the current room using `enter()`.
- Retrieves the current room using `get_current()`.
- Checks whether a specified room is current using `is_current()`.
- Clears the current room using `clear()`.
- Validates manager references and room-name argument types.

3. Data Structure

A Room Manager instance contains the following field:

Field	Type	Initial Value	Description
`current_room`	`string` or `nil`	`nil`	The name of the currently registered room.

Example:

```
local RoomManager = require
"scripts.gameplay.room_manager"

local room_manager = RoomManager.new()
```

Initial state:

```
{
    current_room = nil
}
```

After entering a room named `"forgotten_forest"`:

```
{
    current_room = "forgotten_forest"
}
```

4. API Reference

4.1. `RoomManager.new()`

Creates a new Room Manager instance.

Parameters: None

Returns: A new manager table with `current_room` initialized to `nil`.

Example:

```
local room_manager = RoomManager.new()
```

4.2. `RoomManager.enter(manager, room_name)`

Sets the current room name.

Parameters:

- `manager` — The Room Manager instance.
- `room_name` (`string`) — The name to register as the current room.

Returns: Nothing.

Assertions:

- `manager` must not be `nil`.
- `room_name` must be a string.

Example:

```
RoomManager.enter(room_manager,
"forgotten_forest")
```

Implementation note: This function replaces the previously stored room name immediately. It does not verify that the room exists or perform an actual room transition.

4.3. `RoomManager.get_current(manager)`

Returns the currently stored room name.

Parameters:

- `manager` — The Room Manager instance.

Returns:

- `string` — The current room name, if one has been registered.
- `nil` — If no room is currently registered.

Assertions:

- `manager` must not be `nil`.

Example:

```
local current_room =
RoomManager.get_current(room_manager)

if current_room then
    print("Current room: " .. current_room)
end
```

4.4. `RoomManager.is_current(manager, room_name)`

Checks whether the specified room name matches the currently stored room name.

Parameters:

- `manager` — The Room Manager instance.
- `room_name` (`string`) — The room name to check.

Returns: `boolean`

- `true` if `manager.current_room` equals `room_name`.
- `false` otherwise, including when no room is registered.

Assertions:

- `manager` must not be `nil`.
- `room_name` must be a string.

Example:

```
if RoomManager.is_current(room_manager,
"forgotten_forest") then
```

```
    print("The player is registered in the
Forgotten Forest.")
end
```

The function performs an exact string comparison. It does not normalize room names or check room identity through Defold.

4.5. `RoomManager.clear(manager)`

Clears the stored current room name.

Parameters:

- `manager` — The Room Manager instance.

Returns: Nothing.

Assertions:

- `manager` must not be `nil`.

Example:

```
RoomManager.clear(room_manager)
```

After calling this function, `get_current()` returns `nil`, and `is_current()` returns `false` for any string room name.

5. Usage Example

```
local RoomManager = require
"scripts.gameplay.room_manager"

local room_manager = RoomManager.new()

-- Register the current room.
RoomManager.enter(room_manager,
"forgotten_forest")

-- Retrieve the current room.
local current_room =
RoomManager.get_current(room_manager)
print(current_room) -- forgotten_forest

-- Check the current room.
if RoomManager.is_current(room_manager,
"forgotten_forest") then
    print("The player is in the Forgotten
Forest.")
end

-- Clear the current room.
RoomManager.clear(room_manager)

print(RoomManager.get_current(room_manager)) --
nil
```

6. Implementation Notes

- **State storage:** The system stores one room name per manager instance.
- **Direct assignment:** `enter()` overwrites the existing room name without retaining the previous room.
- **Room-name validation:** The system verifies that room names are strings but does not reject empty strings.
- **No room validation:** Room names are not checked against existing Defold collections, game objects, or room definitions.
- **No transition logic:** Registering a room does not load, unload, or switch any actual game content.
- **No persistence:** The current room is stored in memory and is not saved to disk.
- **Independent instances:** Each manager created with `new()` maintains its own `current_room` field.

7. Limitations

The current implementation does not provide:

- Room loading or unloading.
- Scene or collection transitions.
- Validation against registered rooms.
- Transition events or callbacks.
- Previous-room tracking.
- Integration with the Checkpoint or Respawn Systems.
- Automatic synchronization with the actual game world.

These responsibilities would require additional implementation or integration with other systems.

8. Quick Reference

Function	Purpose
`new()`	Creates a Room Manager instance.
`enter(manager, room_name)`	Sets the current room name.
`get_current(manager)`	Returns the current room name or `nil`.
`is_current(manager, room_name)`	Checks whether a room name matches the current room.
`clear(manager)`	Resets the current room name to `nil`.

9. Usage Guidelines

- Create a manager instance for the context in which room tracking is needed.
- Use `enter()` to update the registered room name when the surrounding game logic determines that the current room has changed.
- Use `get_current()` when other systems need the stored room name.
- Use `is_current()` for simple room-name checks.
- Use `clear()` when the current room reference should no longer be available.
- Do not treat the stored room name as proof that a room has been loaded or that a transition has completed.

10. Summary

The Room Manager is a lightweight state-tracking module that stores the name of the current room. Its responsibility is limited to maintaining and querying that value; actual room loading, transitions, and world-state synchronization remain outside its scope.

Stats System

File: `scripts/gameplay/stats.lua`
Category: Gameplay Systems
Status: Implemented

1. Overview

The Stats System stores and manages named numerical values.

It provides functions to create a stats instance, set and retrieve values, modify existing values, check whether a stat exists, define optional minimum and maximum limits, and remove a stat along with its associated limits.

The system is generic and can be used to represent gameplay values such as attack power, movement speed, stamina, or other numerical attributes.

It does not define specific stats or determine how they affect gameplay.

2. Features

- Stores numerical stats under string names.
- Sets and retrieves stat values.
- Modifies existing stat values by a specified amount.
- Supports optional minimum and maximum limits.
- Applies configured limits when modifying a stat.
- Checks whether a stat exists.
- Removes a stat and its associated limits.
- Validates argument types and the existence of stats where required.

3. Data Structure

A Stats instance contains a `values` table.

```
{
    values = {
        attack = 10,
        attack_max = 20,
        attack_min = 0
    }
}
```

Field	Type	Description
`values`	`table`	Stores stat values and their optional minimum and maximum limits.
`values[name]`	`number`	The value of a stat.
`values[name .. "_max"]`	`number` or `nil`	The optional maximum limit associated with a stat.
`values[name .. "_min"]`	`number` or `nil`	The optional minimum limit associated with a stat.

Limits are stored in the same table as the stat values. For example, the stat named `"attack"` uses the keys `"attack"`, `"attack_max"`, and `"attack_min"`.

Naming consideration: Because limits use suffixes appended to stat names, names can overlap. For example, a stat named `"speed_max"` could conflict with the maximum-limit key belonging to a stat named `"speed"`. Avoid such naming conflicts when defining stats.

4. API Reference

4.1. `Stats.new()`

Creates a new Stats instance.

Parameters: None

Returns: A new table containing an empty `values` table.

Example:

```
local Stats = require "scripts.gameplay.stats"

local stats = Stats.new()
```

4.2. `Stats.set(stats, name, value)`

Sets a stat to a specified numerical value.

Parameters:

- `stats` — The Stats instance.
- `name` (`string`) — The stat name.
- `value` (`number`) — The value to assign.

Returns: Nothing.

Assertions:

- `stats` must not be `nil`.
- `name` must be a string.
- `value` must be a number.

Example:

```
Stats.set(stats, "attack", 10)
Stats.set(stats, "speed", 5)
```

Implementation note: This function assigns the value directly. It does not enforce any configured minimum or maximum limits. It can also create a new stat if the name does not already exist.

4.3. `Stats.get(stats, name)`

Returns the stored value of a stat.

Parameters:

- `stats` — The Stats instance.
- `name` (`string`) — The stat name.

Returns:

- `number` — The stored value, if the stat exists.
- `nil` — If the stat does not exist.

Assertions:

- `stats` must not be `nil`.
- `name` must be a string.

Example:

```
local attack = Stats.get(stats, "attack")
```

This function does not require the stat to exist.

4.4. `Stats.modify(stats, name, amount)`

Modifies an existing stat by adding a numerical amount to its current value.

Parameters:

- `stats` — The Stats instance.
- `name` (`string`) — The stat name.
- `amount` (`number`) — The amount to add. Negative values can be used to decrease the stat.

Returns: The resulting stat value.

Assertions:

- `stats` must not be `nil`.
- `name` must be a string.
- `amount` must be a number.
- The specified stat must exist.

Behavior:

1. Retrieves the current value.
2. Adds `amount` to the current value.
3. If a maximum limit exists, caps the result at that maximum.
4. If a minimum limit exists, raises the result to that minimum if necessary.
5. Stores and returns the resulting value.

Example:

```
Stats.set(stats, "stamina", 50)
Stats.set_max(stats, "stamina", 100)
Stats.set_min(stats, "stamina", 0)

local new_value = Stats.modify(stats, "stamina",
20)
-- Result: 70

Stats.modify(stats, "stamina", -90)
-- Result: 0
```

Important: The minimum check occurs after the maximum check. If the configured minimum exceeds the configured maximum, the final result can exceed the maximum.

4.5. `Stats.has(stats, name)`

Checks whether a stat value exists.

Parameters:

- `stats` — The Stats instance.
- `name` (`string`) — The stat name.

Returns: `boolean`

- `true` if `stats.values[name]` is not `nil`.
- `false` otherwise.

Assertions:

- `stats` must not be `nil`.
- `name` must be a string.

Example:

```
if Stats.has(stats, "attack") then
    print("Attack stat exists.")
end
```

This checks the existence of the stat value, not whether it has configured limits.

4.6. `Stats.set_max(stats, name, max_value)`

Sets the maximum limit associated with an existing stat.

Parameters:

- `stats` — The Stats instance.
- `name` (`string`) — The name of an existing stat.
- `max_value` (`number`) — The maximum value to store.

Returns: Nothing.

Assertions:

- `stats` must not be `nil`.
- `name` must be a string.
- `max_value` must be a number.
- The specified stat must exist.

Example:

```
Stats.set(stats, "health", 80)
Stats.set_max(stats, "health", 100)
```

Setting the maximum does not immediately change the current stat value. The limit is applied when `modify()` is called.

4.7. `Stats.get_max(stats, name)`

Returns the configured maximum limit for a stat.

Parameters:

- `stats` — The Stats instance.
- `name` (`string`) — The stat name.

Returns:

- `number` — The stored maximum limit.
- `nil` — If no maximum limit is configured.

Assertions:

- `stats` must not be `nil`.
- `name` must be a string.

Example:

```
local max_health = Stats.get_max(stats,
"health")
```

The function does not require the stat itself to exist.

4.8. `Stats.set_min(stats, name, min_value)`

Sets the minimum limit associated with an existing stat.

Parameters:

- `stats` — The Stats instance.
- `name` (`string`) — The name of an existing stat.
- `min_value` (`number`) — The minimum value to store.

Returns: Nothing.

Assertions:

- `stats` must not be `nil`.
- `name` must be a string.
- `min_value` must be a number.
- The specified stat must exist.

Example:

```
Stats.set(stats, "stamina", 50)
Stats.set_min(stats, "stamina", 0)
```

Setting the minimum does not immediately change the current stat value. The limit is applied when `modify()` is called.

4.9. `Stats.get_min(stats, name)`

Returns the configured minimum limit for a stat.

Parameters:

- `stats` — The Stats instance.
- `name` (`string`) — The stat name.

Returns:

- `number` — The stored minimum limit.
- `nil` — If no minimum limit is configured.

Assertions:

- `stats` must not be `nil`.
- `name` must be a string.

Example:

```
local min_stamina = Stats.get_min(stats,
"stamina")
```

The function does not require the stat itself to exist.

4.10. `Stats.remove(stats, name)`

Removes a stat and its associated minimum and maximum limits.

Parameters:

- `stats` — The Stats instance.
- `name` (`string`) — The stat name.

Returns: Nothing.

Assertions:

- `stats` must not be `nil`.
- `name` must be a string.

Example:

```
Stats.remove(stats, "attack")
```

This removes the following keys:

- `attack`
- `attack_min`
- `attack_max`

The function does not require the stat to exist. Removing a nonexistent stat is therefore harmless unless its name overlaps with another stat's stored limit key.

5. Usage Example

```
local Stats = require "scripts.gameplay.stats"

local stats = Stats.new()

-- Define stats.
Stats.set(stats, "attack", 10)
Stats.set(stats, "stamina", 50)

-- Configure limits.
Stats.set_min(stats, "attack", 1)
Stats.set_max(stats, "attack", 25)

Stats.set_min(stats, "stamina", 0)
Stats.set_max(stats, "stamina", 100)

-- Read values.
print(Stats.get(stats, "attack")) -- 10

-- Increase attack.
```

```
Stats.modify(stats, "attack", 5)
print(Stats.get(stats, "attack")) -- 15

-- Decrease stamina.
Stats.modify(stats, "stamina", -20)
print(Stats.get(stats, "stamina")) -- 30

-- Check whether a stat exists.
print(Stats.has(stats, "stamina")) -- true

-- Read configured limits.
print(Stats.get_min(stats, "stamina")) -- 0
print(Stats.get_max(stats, "stamina")) -- 100

-- Remove a stat and its limits.
Stats.remove(stats, "attack")
print(Stats.has(stats, "attack")) -- false
```

6. Implementation Notes

- **Generic storage:** The system does not define any specific stat names or gameplay meanings.
- **Numeric values:** Stat values, minimum limits, maximum limits, and modification amounts must have Lua's `number` type.
- **Optional limits:** Stats can exist without either limit.
- **Limits apply during modification:** `modify()` enforces configured limits, while `set()` does not.
- **No automatic correction:** Setting a limit does not immediately clamp the current value.
- **No limit-order validation:** The system does not verify that a minimum is less than or equal to a maximum.
- **No deletion return value:** `remove()` does not report whether a stat existed before removal.
- **No persistence:** The system stores values in memory and does not save or load them.
- **No gameplay integration:** The system does not automatically connect stats to combat, movement, health, or other gameplay behavior.

7. Limitations

The current implementation does not provide:

- Stat definitions or predefined stat types.
- Automatic enforcement of limits during direct assignment.
- Validation that minimum and maximum limits are logically consistent.
- Stat-change callbacks or events.
- Derived stats or automatic calculations between stats.
- Modifiers from equipment, abilities, buffs, or debuffs.
- Persistence or synchronization with other systems.
- Validation that numerical values are finite or integral.

8. Quick Reference

Function	Purpose
`new()`	Creates an empty Stats instance.
`set(stats, name, value)`	Assigns a numerical value.
`get(stats, name)`	Retrieves a stat value or `nil`.
`modify(stats, name, amount)`	Modifies an existing stat and applies configured limits.
`has(stats, name)`	Checks whether a stat exists.
`set_max(stats, name, max_value)`	Sets a stat's maximum limit.
`get_max(stats, name)`	Retrieves the maximum limit or `nil`.
`set_min(stats, name, min_value)`	Sets a stat's minimum limit.
`get_min(stats, name)`	Retrieves the minimum limit or `nil`.
`remove(stats, name)`	Removes a stat and its associated limits.

9. Usage Guidelines

- Use consistent, descriptive names for stats.
- Avoid naming a stat with a suffix that conflicts with another stat's `_min` or `_max` key.

- Configure limits before relying on `modify()` to enforce them.
- Use `modify()` when the change should respect configured limits.
- Use `set()` when direct assignment is intended and limit enforcement is not required.
- Ensure the minimum and maximum limits are consistent.
- Treat missing values and missing limits as distinct cases: `get()` and the limit getters can return `nil`.

10. Summary

The Stats System provides generic numerical stat storage with optional minimum and maximum limits. It supports direct assignment, retrieval, bounded modification, existence checks, and removal.

Its primary responsibility is to manage stat values. Defining their meaning, enforcing game-specific rules, and connecting them to gameplay behavior remain the responsibility of other systems.

Gameplay Object System

File: `scripts/gameplay/gameplay_object.lua`
Category: Gameplay Systems
Status: Implemented

1. Overview

The Gameplay Object System provides a simple container for storing and managing components under string-based names.

Each Gameplay Object instance maintains a `components` table, allowing components to be added, retrieved, checked, removed, or cleared.

The system does not define component behavior or require components to follow a particular structure. A component can be any non-`nil` Lua value, although tables are a natural choice for structured component data.

2. Features

- Creates Gameplay Object instances.
- Stores components under string names.
- Retrieves components by name.
- Checks whether a named component exists.
- Removes individual components.
- Clears all stored components.
- Validates required arguments.

3. Data Structure

Each Gameplay Object instance contains a single field:

Field	Type	Description
`components`	`table`	Maps component names to their stored values.

Example:

```
{
    components = {
        health = {},
        inventory = {},
        stats = {}
    }
}
```

The names in this example are illustrative. The system does not automatically create or initialize Health, Inventory, or Stats components.

4. API Reference

4.1. `GameplayObject.new()`

Creates a new Gameplay Object instance.

Parameters: None

Returns: A new table containing an empty `components` table.

Example:

```
local GameplayObject = require
"scripts.gameplay.gameplay_object"

local object = GameplayObject.new()
```

4.2. `GameplayObject.add(gameplay_object, name, component)`

Adds a component or assigns a new value to an existing component name.

Parameters:

- `gameplay_object` — The Gameplay Object instance.
- `name` (`string`) — The name under which the component is stored.
- `component` — Any non-`nil` value to store.

Returns: Nothing.

Assertions:

- `gameplay_object` must not be `nil`.
- `name` must be a string.
- `component` must not be `nil`.

Example:

```
local health = {
    current = 100,
    max = 100
}

GameplayObject.add(object, "health", health)
```

Implementation note: If a component already exists under the specified name, its stored value is replaced. The function does not prevent duplicate names or merge components.

4.3. `GameplayObject.get(gameplay_object, name)`

Retrieves the value stored under a component name.

Parameters:

- `gameplay_object` — The Gameplay Object instance.
- `name` (`string`) — The component name.

Returns:

- The stored component value, if present.
- `nil` if no value is stored under that name.

Assertions:

- `gameplay_object` must not be `nil`.
- `name` must be a string.

Example:

```
local health = GameplayObject.get(object,
"health")

if health then
    print(health.current)
end
```

The function does not require the component to exist.

4.4. `GameplayObject.has(gameplay_object, name)`

Checks whether a component is stored under the specified name.

Parameters:

- `gameplay_object` — The Gameplay Object instance.
- `name` (`string`) — The component name.

Returns: `boolean`

- `true` if the stored value is not `nil`.
- `false` otherwise.

Assertions:

- `gameplay_object` must not be `nil`.
- `name` must be a string.

Example:

```
if GameplayObject.has(object, "health") then
    print("The object has a health component.")
end
```

4.5. `GameplayObject.remove(gameplay_object, name)`

Removes the component stored under the specified name.

Parameters:

- `gameplay_object` — The Gameplay Object instance.
- `name` (`string`) — The component name.

Returns: Nothing.

Assertions:

- `gameplay_object` must not be `nil`.
- `name` must be a string.

Example:

```
GameplayObject.remove(object, "health")
```

If the name does not exist, the operation has no effect. The function does not require the component to exist before removal.

4.6. `GameplayObject.clear(gameplay_object)`

Removes all components from the Gameplay Object by replacing its `components` table with a new empty table.

Parameters:

- `gameplay_object` — The Gameplay Object instance.

Returns: Nothing.

Assertions:

- `gameplay_object` must not be `nil`.

Example:

```
GameplayObject.clear(object)
```

After this call, the object contains an empty `components` table.

Implementation note: The function replaces the table rather than deleting its entries individually. Any external reference to the previous table remains valid and can still access its old contents.

5. Usage Example

```
local GameplayObject = require
"scripts.gameplay.gameplay_object"

local object = GameplayObject.new()

-- Create example component data.
local health = {
    current = 100,
    max = 100
}
```

```
local stats = {
    attack = 10,
    defense = 5
}

-- Add components.
GameplayObject.add(object, "health", health)
GameplayObject.add(object, "stats", stats)

-- Check for a component.
if GameplayObject.has(object, "health") then
    print("Health component found.")
end

-- Retrieve and use a component.
local stored_health = GameplayObject.get(object,
"health")

if stored_health then
    stored_health.current =
stored_health.current - 10
    print(stored_health.current) -- 90
end

-- Remove one component.
GameplayObject.remove(object, "stats")

-- Clear all remaining components.
GameplayObject.clear(object)
```

The example uses ordinary Lua tables as component data. The Gameplay Object System itself does not define or manage the internal structure of those tables.

6. Implementation Notes

- **Name-based storage:** Components are stored in a Lua table using string keys.
- **Unrestricted component values:** The system accepts any non-`nil` value as a component; it does not validate a component's type or interface.
- **Replacement behavior:** Adding a component under an existing name overwrites the previous value.
- **Reference semantics:** Retrieving a table-valued component returns the same table reference, not a copy. Changes to that table affect the stored component.
- **No automatic initialization:** Components must be created or obtained elsewhere before being added.
- **No component lifecycle management:** Removing or clearing components does not call cleanup functions or destroy any associated objects.
- **No automatic behavior:** The system does not update components, dispatch events, or execute component methods.
- **No persistence:** Component data is not saved to disk.

7. Limitations

The current implementation does not provide:

- Component-specific type validation.
- Automatic component creation or initialization.
- Component dependency management.
- Update or lifecycle callbacks.
- Automatic cleanup when components are removed.
- Entity identification or world registration.
- Integration with Defold game objects or collections.
- Serialization or persistence.

These features would require additional implementation or integration with other systems.

8. Quick Reference

Function	Purpose

`new()`	Creates a Gameplay Object with an empty component table.
`add(gameplay_object, name, component)`	Stores or replaces a component.
`get(gameplay_object, name)`	Retrieves a component or returns `nil`.
`has(gameplay_object, name)`	Checks whether a component exists.
`remove(gameplay_object, name)`	Removes a named component.
`clear(gameplay_object)`	Replaces the component table with an empty table.

9. Usage Guidelines

- Use consistent component names across your project.
- Add components only after their data or instances have been initialized.
- Use `has()` when component existence must be checked before use.
- Use `get()` when retrieving a component that may not exist.
- Remember that adding a component under an existing name replaces the previous value.
- Do not assume that removing a component performs cleanup or destroys associated resources.
- Keep component behavior in the relevant systems unless this module is deliberately extended to manage it.

10. Summary

The Gameplay Object System is a lightweight, name-based component container. It provides basic storage and retrieval operations while leaving component creation, behavior, lifecycle management, and integration with the game world to other systems.